Software Engineering and Continuous Integration (SENC / SWEC)

Vorlesung an der

Fachhochschule der Wirtschaft (FHDW)

in Bergisch Gladbach bzw. Mettmann

Dozent: Prof. Dr. Wolfgang Kahnert

Handouts: Nur zum persönlichen Gebrauch!

Organisatorisches

- Vorstellung / „Who is who?“

Zu meiner Person: Prof. Dr. Wolfgang Kahnert

wolfgang.kahnert@fhdw.de

Tel.: 02202 / 9527 - 384

Sprechzeiten: nach Vereinbarung

Organisatorisches

- „Präsenz“- (inkl. Online-)Veranstaltungen:
  - insbes. Präsentationen mit Dialogen und Diskussionen, Übungen
  - Fragen, Ergänzungen etc.: jederzeit möglich und erwünscht!
  - Auch Online-Vorlesungen sind Echtzeit- / synchrone Vorlesungen
  - Hybrid-Vorlesungen (= Vorlesungen in Präsenz und gleichzeitig online übertragen mittels Teams):
    Maßgeblich ist die Präsenz-Vorlesung vor Ort!
    Die Teilnahme an der *Präsenz*-Vorlesung ist dringend empfohlen!
  - Video-Aufzeichnungen der Vorlesungen sind nicht erlaubt!
  - Handouts: Nur zur persönlichen Verwendung für die Teilnehmer der entsprechenden Vorlesung! Eine Weitergabe der (digitalen oder analogen) Handouts u.Ä. an Dritte ist nicht erlaubt!
  - Bei Online- / Teams-Vorlesungen: Außer bei Wortmeldungen bitte das Mikrofon deaktiviert lassen!

Organisatorisches

- Formalia:
  - SENC / SWEC
  - 40 Vorlesungs-Stunden („Kontaktstunden“) + 85 Selbststudium-Stunden
  - 5 ECTS

- Prüfungsleistung: 1 Klausur (90 Min.)

- Notation: ◄► bedeutet „versus“

Inhalte (Curriculum)

- Einführung
- Vorgehensmodelle
- Requirements Engineering
- Software-Tests
- Spezifikationstechniken

Literatur

- Gemäß Curriculum:
  - Sommerville: Software Engineering
  - Sommerville: Modernes Software Engineering
  - Ludewig, Lichter: Software Engineering

- Erg.:
  - Balzert, Helmut: Lehrbuch der Softwaretechnik:
    - Basiskonzepte und Requirements Engineering
    - Softwaremanagement
    - Entwurf, Implementierung, Installation, Betrieb
  - Grechenig et al.: Softwaretechnik
  - Oestereich, Scheithauer: Analyse und Design mit der UML 2.[x]
  - Pohl, Rupp: Basiswissen Requirements Engineering
  - Rupp & die SOPHISTen: Requirements-Engineering und -Management
  - IEEE: SWEBOK

Erg. Hinweise

- Die Abbildungen sind hauptsächlich entnommen aus:
  - /Bal/: H. Balzert: Präsentationen zur Software-Technik 1, Spektrum, 1998
  - /Som/: I. Sommerville: Software Engineering, Pearson Studium, 2007
  - /Lud/: J. Ludewig, H. Lichter: Software Engineering, dpunkt.verlag, 2007
  - /Gre/: T. Grechenig et al.: Softwaretechnik, Pearson Studium, 2010

1.1) Begriffsklärungen

- Software Engineering =
  - „Software-Technik: Zielorientierte Bereitstellung und systematische Verwendung von Prinzipen, Methoden und Werkzeugen für die arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung von umfangreichen Software-Systemen. Zielorientiert bedeutet die Berücksichtigung z.B. von Kosten, Zeit, Qualität.“ *(Balzert, a.a.O.)*
  - „…. das Entwerfen, Herstellen u. Implementieren von Software sowie die ingenieurwissenschaftliche Disziplin, die sich mit Methoden u. Verfahren der damit verbundenen Problemstellungen befasst.“
    (Duden / Das Fremdwörterbuch, 9. Aufl.)
  - „(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.
    (2) The study of approaches as in (1).“
    (IEEE Standard Glossary of Software Engineering Terminology, IEEE Std. 610.12-1990)

1.1) Begriffsklärungen

Software Engineering (Fortsetzg.)

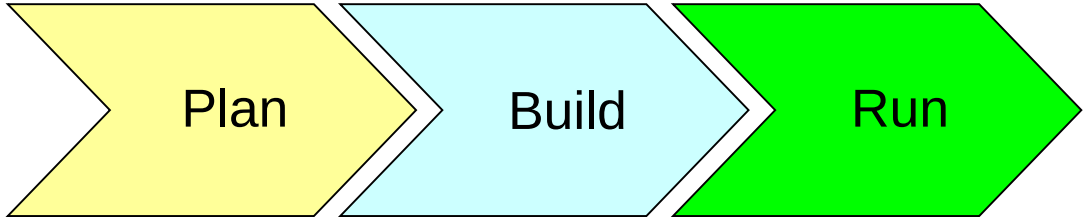
- Fazit: Software Engineering: technische Disziplin
  - bzgl. aller Phasen (von den ersten Ideen und „Analysen“ über die Beschaffung bzw. Entwicklung, die Einführung, den Betrieb bis zur Wartung bzw. Ablösung)
  - und bzgl. aller Komponenten (inkl. Methodiken, Standards etc.)
  - für einen erfolgreichen „Software-Einsatz“ (im o.g., weitesten Sinne)
- Synonym: Software-Technik
- Abgrenzung: System Engineering

1.2) Motivation

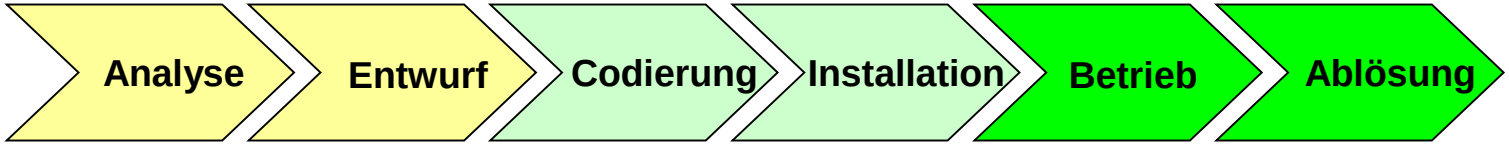
- Motivation für Software Engineering:
  - Abhängigkeit von automatisierter Informationsverarbeitung mittels Computer-Systemen: in Wirtschaft und Öffentl. Verwaltung existentiell; im Privatleben zunehmend
  - Detail: Weltweite Daten-Menge: Quelle: Statista 2024 / Schätzungen / u.A.
    - 2010: ca. 2 ZByte
    - 2015: ca. 16 ZByte
    - 2020: ca. 64 ZByte
    - 2023: ca. 126 ZByte
    - 2024: ca. 154 ZByte
    - 2027: ca. 284 ZByte
  - „Software Crisis“
- Fazit: Software erfordert eine *professionelle* Erstellung etc., Software-Engineering ist ein MUSS!

1.3) Software-Lebenszyklus

- Grob-Modell:



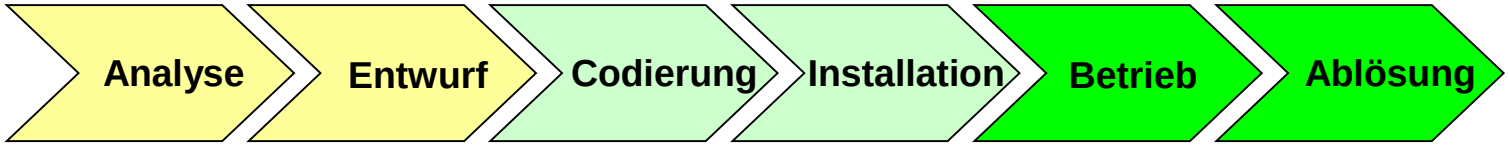
- Verfeinerung (auch modifiz. Modelle oder Bezeichnungen üblich):



- Hinweis: ISO/IEC 12207: Software Life Cycle Processes

1.3) Software-Lebenszyklus

- Einige wichtige Aspekte des SWE:



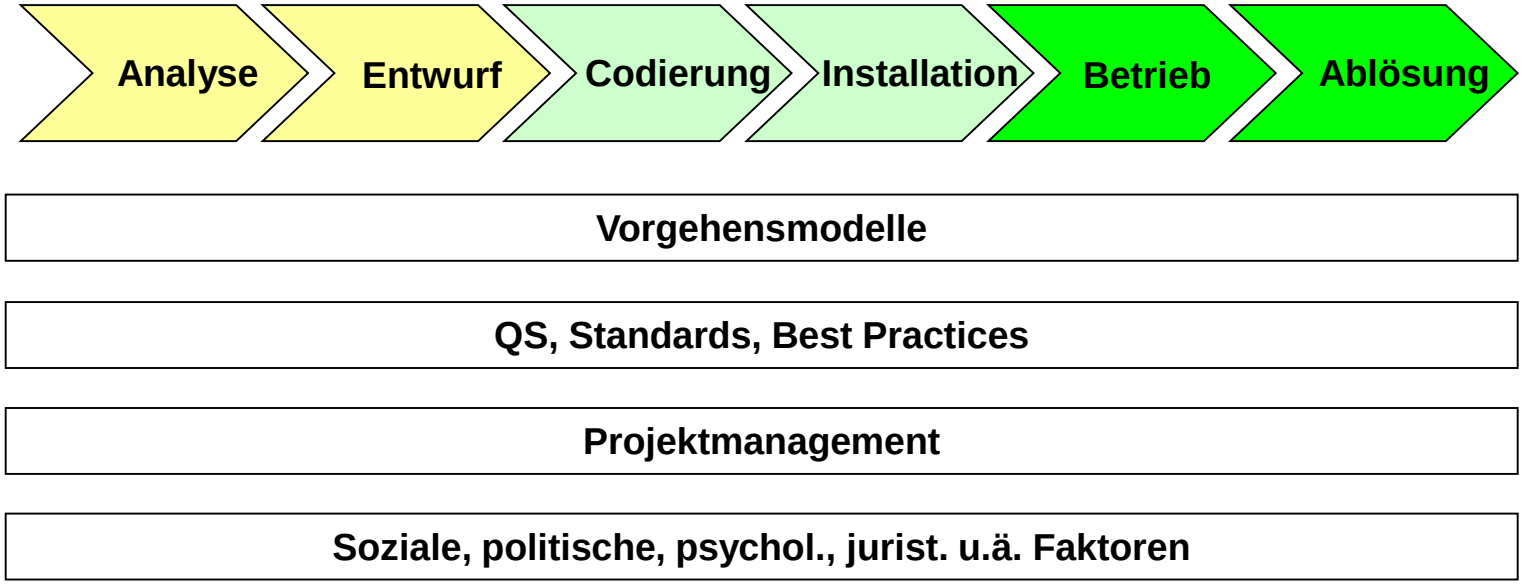
- Problemanalyse
- Requirements Engineering
- Machbarkeitsanalyse
- Aufwandsschätzung
- Kosten-Nutzen-Analyse
- GO-or-NOGO-Entscheidung
- Make-or-Buy-Entscheidung
- Grob- und Fein-Entwürfe
- Lasten- und Pflichtenheft
- Design
- Beauftragung

- Spezifikationen
- Struktogramm-Notation u.A.
- OOSE
- UML
- Codierung i.e.S.
- Integration
- Tests
- Verifikation bzw. Validierung
- Abnahme
- CASE, SEU, IDE

- Mitarbeiterschulung
- Roll-Out
- Systembetrieb
- Wartung, Support und Pflege
- Migration auf Neusystem
- Ablösung des Altsystems

1.3) Software-Lebenszyklus

- Einige wichtige Aspekte des SWE (Fortsetzg.):



1.3) Software-Lebenszyklus

- Einschub: Aufwands- / Kostenverteilung

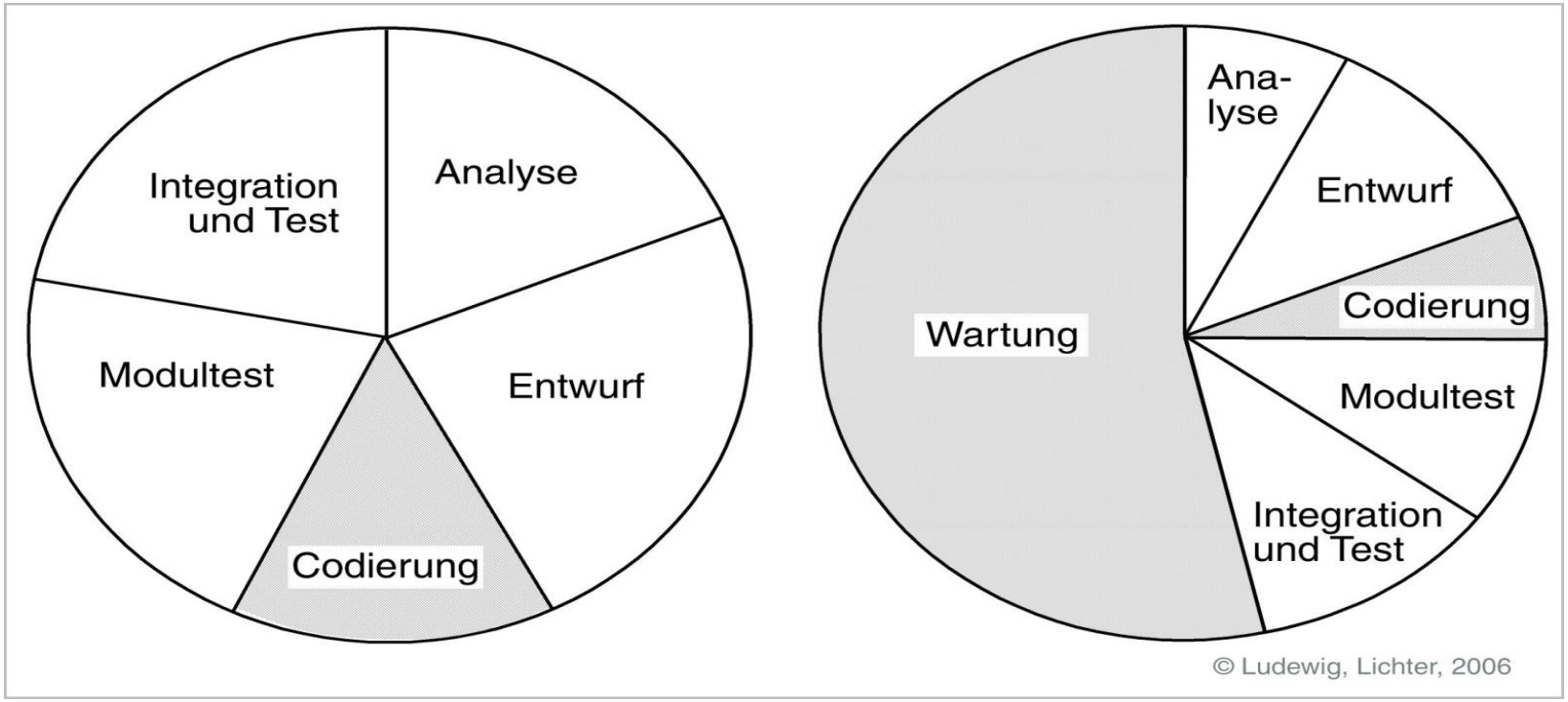


Abb. aus /Lud/

1.3) Einführung: Software-Lebenszyklus

- In allen Phasen lauern Probleme:

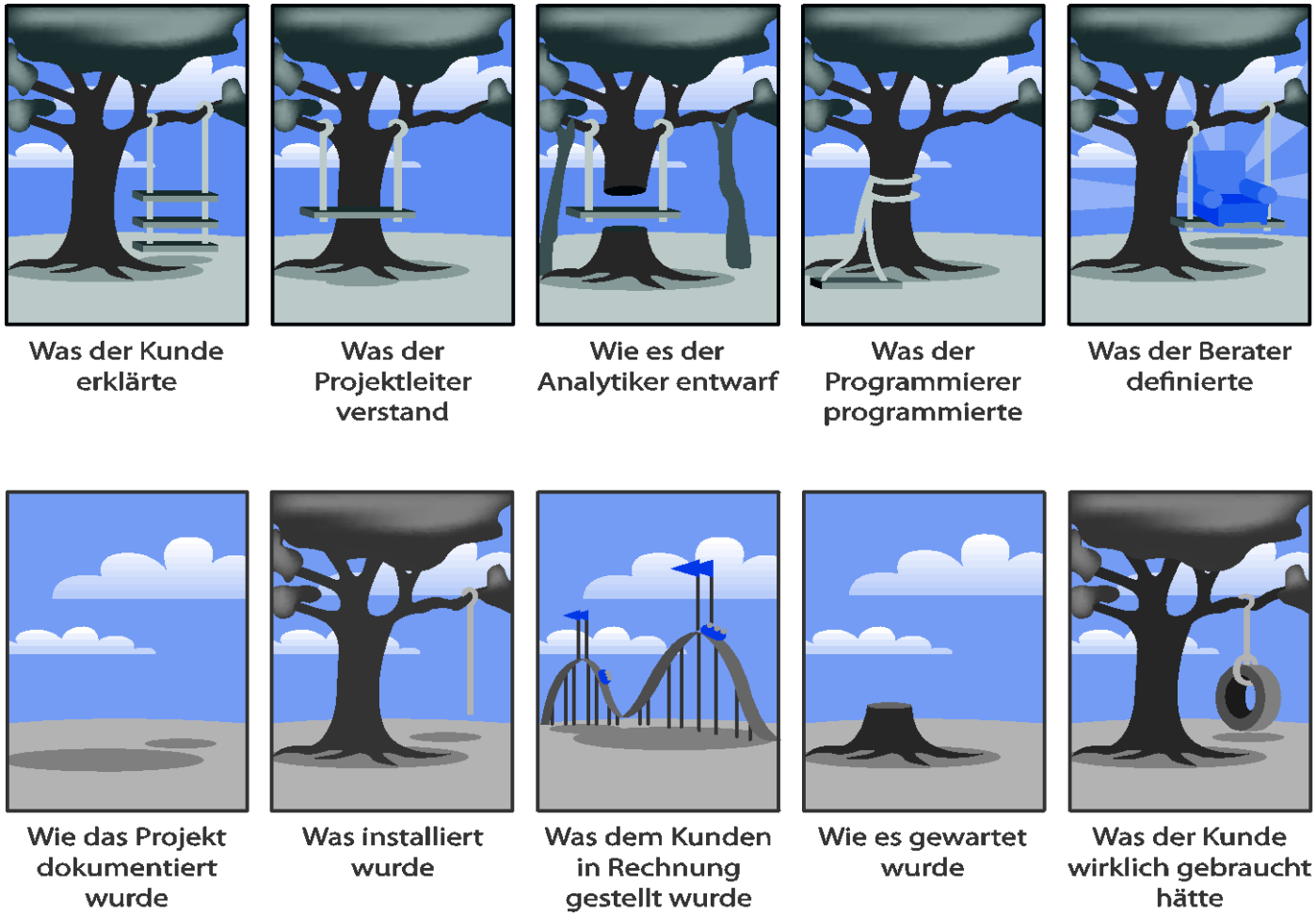


Abb. aus /Gre/

2) Vorgehensmodelle / 2.1) Vorgehensmodelle: Allg.

- Vorgehensmodelle:
  - = allg. / abstraktes Schema, dass das generelle Vorgehen* / Prozesse bei der Entwicklung einer Software festlegt
  - *Vorgehen umfaßt:
    - Phasen (Reihenfolge)
    - Aktivitäten
    - Ergebnisse
    - Rollen
  - Es gibt sehr viele verschiedene Vorgehensmodelle, aber best. Grund-Typen

2.2) Vorgehensmodelle: Begriffe

- Iterative Entwicklung = Entwicklung des Produktes in mehreren Schritten (Teil-Projekten) mit iterativer Korrektur bzw. Weiterentwicklung des (Zwischen-) Produktes; pro Iterationsschritt: Analyse, Entwurf, Code, Test
- Inkrementelle Entwicklung* = zunächst Entwicklung des (einsetzbaren!) Kern-Produktes, dann Erweiterungen in Form von Ausbaustufen / Inkrementen
  * Achtung: auch anderer Sprachgebrauch verbreitet!
- Treppenmodell = Inkrementelles Modell mit sich überlappenden Ausbaustufen:

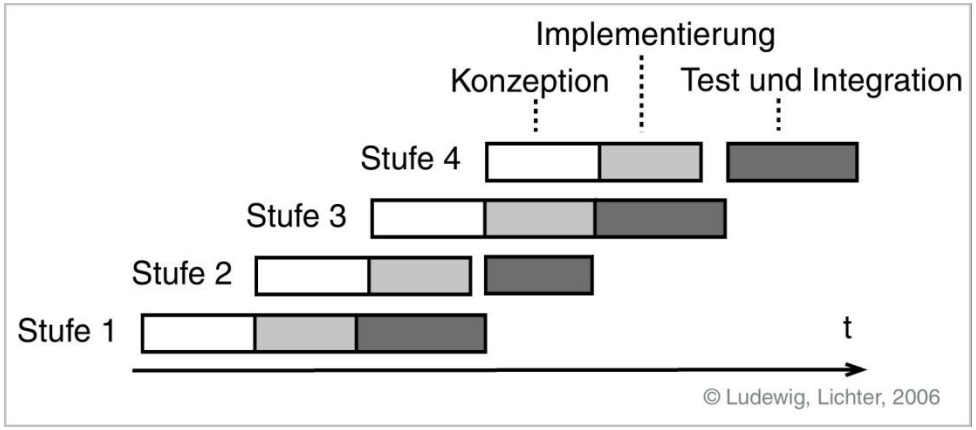


Abb. aus /Lud/

2.2) Vorgehensmodelle: Begriffe

Verifikation und Validierung

- Verifikation = Überprüfung der Übereinstimmung zwischen der Software und ihrer Spezifikation („Wird das Produkt *richtig entwickelt*?“)

◀▶

- Validierung / Validation = Überprüfung zwischen der Übereinstimmung der Software mit dem Einsatzzweck des (bzw. Wert für den) Auftraggeber („Wird das *richtige Produkt* entwickelt?“)

2.3) Vorgehensmodelle: Wasserfallmodell

- Wasserfallmodell:

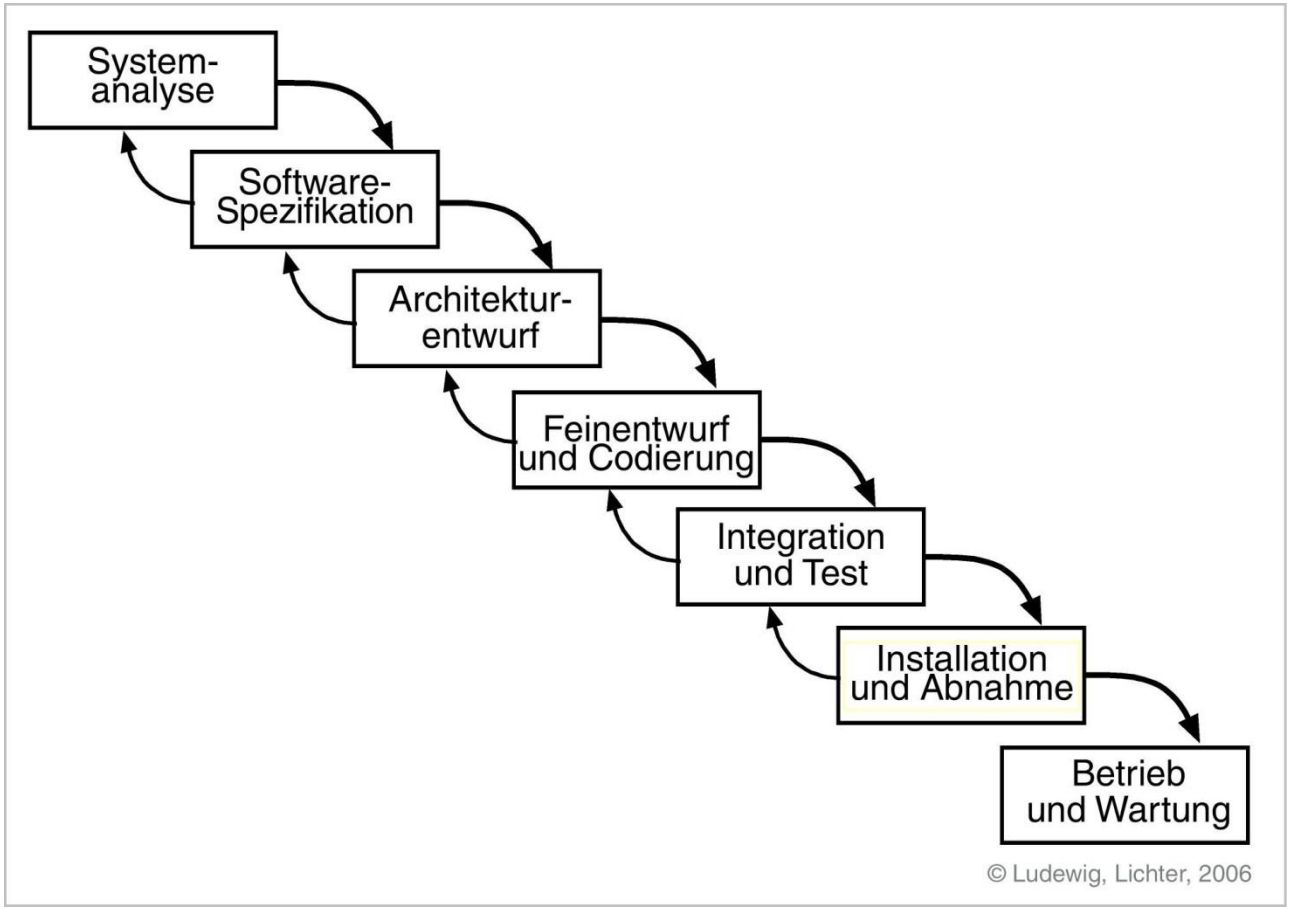


Abb. aus /Lud/

2.3) Vorgehensmodelle: Wasserfallmodell

- Royce, 1970
- Weit verbreitet, intuitiv
- Linear / sequentiell
- Alle Phasen + Aktivitäten sind jeweils vollständig durchzuführen
- Ergebnis jeder Phase = Dokument → dokumenten-getriebenes Vorgehen
- Top-down-Vorgehen
- „Rückkopplungen“: als „Zyklus“ beschränkt auf die jeweils vorangegangene Phase!
- Beteiligung des Auftraggebers: beschränkt auf die Festlegung der Anforderungen

2.3) Vorgehensmodelle: Wasserfallmodell

Vorteile

- Einfache Verständlichkeit und Anwendbarkeit
- Zwang zum strukturierten Vorgehen: Definieren ⇒ Entwerfen ⇒ Codieren etc.
- Zwang zu fixierten Anforderungen (Vermeidung von „Moving Targets“ u.Ä.), Schnittstellen etc.
- Zwang zu Meilensteinen und „auslieferbaren“ Ergebnissen
- Klarer Prozessablauf

2.3) Vorgehensmodelle: Wasserfallmodell

Nachteile

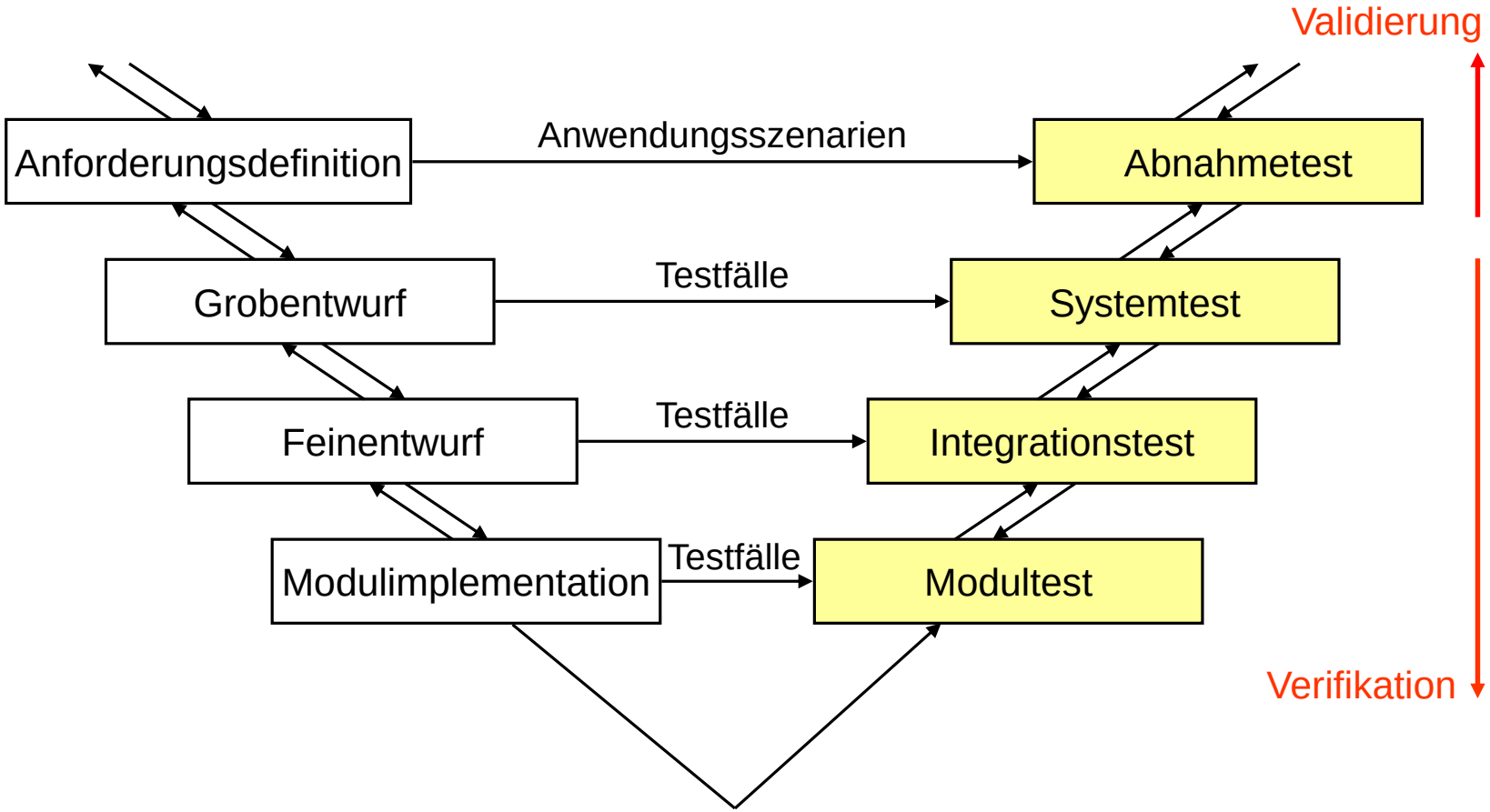
- Diskrepanz zwischen Modell und Realität: z.B.
  - oftmals sind Auftraggeber-Anforderungen so früh nicht vollständig fixierbar oder eindeutig geklärt
  - Tests und Auslieferung erst in späten Phasen ⇒ Probleme erst spät erkennbar und nur mit großem Aufwand* lösbar
  - *ggf. Rückkopplung über mehrere Phasen
  - *Änderung von Ergebnis-Dokumenten
  - Schwierigkeit für ein umfassendes Risiko-Management und QS
  - Signifikanter Verwaltungs-“Overhead“

2.3) Vorgehensmodelle: Wasserfallmodell

- Einsatz-Empfehlung:
  - Falls Anforderungen gut fixiert sind
  - Falls Entwicklung weitgehend risikolos ist
  - Insbes. für geringfügigere Updates eines bestehenden Produktes

2.4) Vorgehensmodelle: V-Modelle

- V-Modell:



2.4) Vorgehensmodelle: V-Modelle

V-Modell® XT:

- „„V-Modell“ ist eine eingetragene Wortmarke der Bundesrepublik Deutschland“*
- „Das deutsche Referenzmodell für Systementwicklungsprojekte“*
- Weiterentwicklung des V-Modells 97
- XT = eXtreme Tailoring
- Seit 2005; seit Q1 / 2019: Version 2.3
  (Q3 / 2015: Version 2.0; Q2 2017: Version 2.1; Q2 / 2018: Version 2.2)
- www.cio.bund.de

* Zitate und Abb. aus der Original-Doku „V-Modell XT“ (Hrsg.: Weit e.V.)

2.4) Vorgehensmodelle: V-Modelle

➢ V-Modell® XT (Fortsetzg.):

- Wesentliche Änderungen gegenüber V-Modell 97:
  - Vereinfachtes Tailoring
  - Orientierung an „Produkten“ / Ergebnissen (statt Aktivitäten)
  - „Entscheidungspunkte“
  - Klare Rollen-Trennung Auftraggeber (AG) ◄► Auftragnehmer (AN) ◄► AG/AN
  - Integration von Normen, (Quasi-)Standards u.Ä.
  - Änderung von Bezeichnungen

2.4) Vorgehensmodelle: V-Modelle

V-Modell XT (Fortsetzg.):

- 3 Projekttypen:
  - Systementwicklungsprojekt* AG
  - Systementwicklungsprojekt* AN
  - Systementwicklungsprojekt* AG/AN

  *bzgl. Hardware, Software, Systemintegration, Embedded System, Komplettsystem, dabei konkrete Auswahl / Festlegung: abhängig von Projektrolle und -gegenstand; weitere Differenzierung (nach Auftragsstruktur bzw. Systemlebenszyklus-Ausschnitt) in Projekttypvarianten:

Projekttyp	Systementwicklungsprojekt (AG)	Systementwicklungsprojekt (AN)	Systementwicklungsprojekt (AG/AN)
Projekttypvariante	AG-Projekt mit einem Auftragnehmer	AN-Projekt mit Entwicklung, Weiterentwicklung oder Migration	AG/AN-Projekt mit Entwicklung, Weiterentwicklung oder Migration
	AG-Projekt mit mehreren Auftragnehmern	AN-Projekt mit Wartung und Pflege	AG/AN-Projekt mit Wartung und Pflege

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - Entwicklungsstrategien (des Projekttyps Systementwicklungsprojekt (AN)):
    - *Inkrementelle* Systementwicklung
    - *Komponentenbasierte* Systementwicklung
    - *Prototypische* Systementwicklung
  - Ablaufbausteine:
    - o.g. Entwicklungsstrategien
    - Unterauftrag

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - Produkt = (Zwischen-)Ergebnis (*z.B.* Doku, Modell, Sourcecode)
  - 110 Produkte → 20 “Disziplinen”*
  - Korrespondierend: Produkte und Aktivitäten
  - Beteiligte: Rollen

* (Disziplinen → 3 Bereiche: Management, Entwicklung, AG-AN-Schnittstelle)

2.4) Vorgehensmodelle: V-Modelle

V-Modell XT (Fortsetzg.):

- Disziplinen:
  - 20 Disziplinen → 3 Bereiche: Management, Entwicklung, AG-AN-Schnittstelle
  - Bereich Management (8):
    - Anbahnung und Organisation
    - Planung und Steuerung
    - Risikomanagement
    - Problem- und Änderungsmanagement
    - Konfigurationsmanagement
    - QS
    - Messung und Analyse
    - Berichtswesen

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - Disziplinen (Fortsetzg.):
    - Bereich Entwicklung (9):
      - Systemelemente
      - Systemanalyse
      - Systementwurf
      - Systemspezifikation
      - Logistikelemente
      - IT-Organisation und Betrieb
      - Logistikkonzeption
      - Informationssicherheit und Datenschutz
      - Funktionssicherheit
    - AG-AN-Schnittstelle (3):
      - Ausschreibungs- und Vertragswesen
      - Angebots- und Vertragswesen
      - Lieferung und Abnahme

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - 36 Rollen:
    - Z.B.: Anforderungsanalytiker (AG ◄► AN), SW-Architekt, SW-Entwickler, Change Control Board, Lenkungsausschuss u.v.a.m
    - Spezifikation insbes. bzgl.:
      - Aufgaben und Befugnisse
      - Fähigkeitsprofil
      - Rollenbesetzung
      - Verantwortlichkeiten bzw. Mitwirkungen

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - 28 Vorgehensbausteine:

    = Zusammenfassungen von i.d.R. Produkten, Aktivitäten und Rollen für eine spez. Aufgabe wie z.B. Konfigurationsmanagement; inkl. weiterer Hinweise (Sinn / Zweck, typische Aspekte)

    - Projektmanagement*
    - QS*
    - Konfigurationsmanagement*
    - Problem- und Änderungsmanagement*
    - Kaufmännisches Projektmanagement
    - Multi-Projektmanagement
    - Messung und Analyse
    - Anforderungsfestlegung

    *obligatorisch für jeden Projekttyp!

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - 28 Vorgehensbausteine (Fortsetzung):
    - Vertragsschluss (AG)
    - Vertragsschluss (AN)
    - Systemerstellung
    - SW-Entwicklung
    - HW-Entwicklung
    - Logistikkonzeption
    - Evaluierung von Fertigprodukten
    - Benutzbarkeit und Ergonomie
    - Weiterentwicklung und Migration von Altsystemen

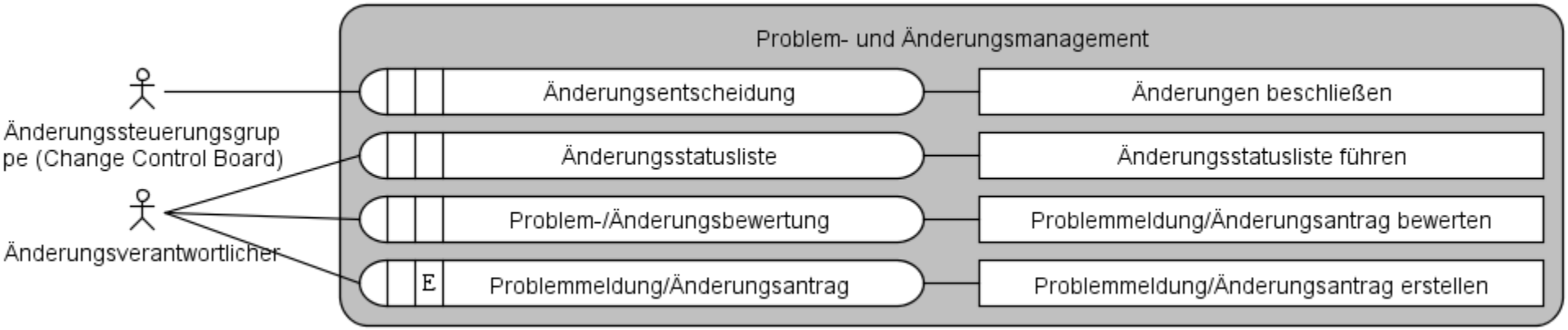
2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - 28 Vorgehensbausteine (Fortsetzung):
    - Informationssicherheit und Datenschutz
    - Informationssicherheit und Datenschutz (AG)
    - Informationssicherheit und Datenschutz (AN)
    - Funktionssicherheit
    - Funktionssicherheit (AG)
    - Funktionssicherheit (AN)
    - Betriebsübergabe
    - Betriebsübergabe (AG)
    - Betriebsübergabe (AN)
    - Lieferung und Abnahme (AG)
    - Lieferung und Abnahme (AN)

2.4) Vorgehensmodelle: V-Modelle

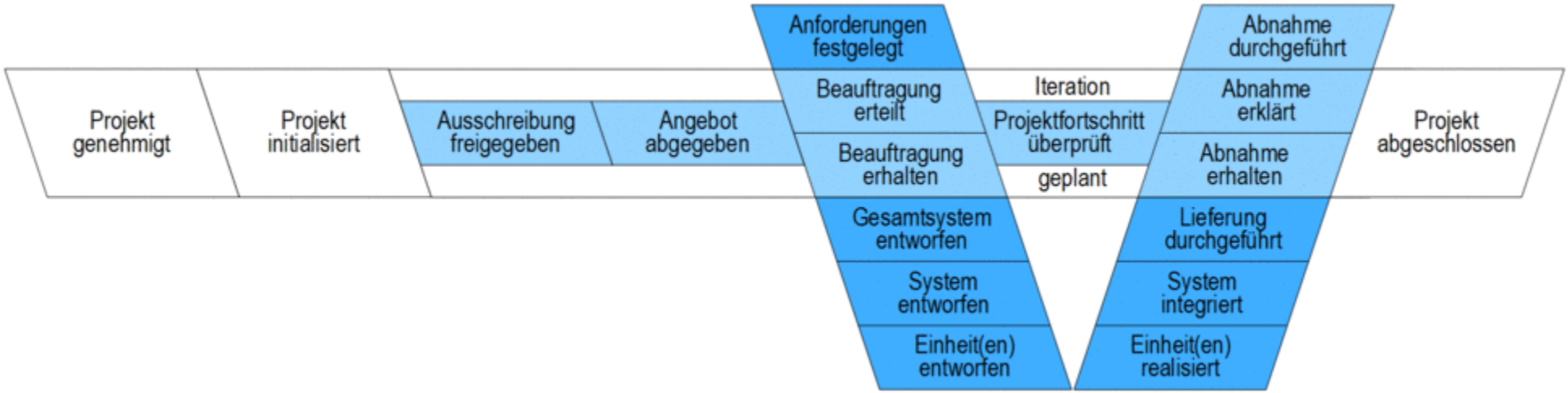
- V-Modell XT (Fortsetzg.):

  Bsp.: Vorgehensbaustein Problem- und Änderungsmanagement



2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - 21 Entscheidungspunkte: bzgl.
    - Meilenstein
    - Produkt
    - Entscheidung



2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - Projektdurchführungsstrategien:
    - = Festlegung der Reihenfolge der zu erstellenden Produkte, Aktivitäten und Entscheidungspunkte
  - Tailoring:
    - Festlegung der Vorgehensbausteine und der Projektdurchführungsstrategie in Abhängigkeit von:
      - Projektgegenstand (z.B. Hard- ◄► Software)
      - Rolle: AG ◄► AN ◄► AG/AN
      - System-Lebenszyklus-Ausschnitt (Entwicklung ◄► Wartung ◄► Migration)
      - Einsatz von Fertigprodukten
      - Risikoeinstufung

      u.a. Projektmerkmalen
    - Automatisierung mittels des Projektassistenten

2.4) Vorgehensmodelle: V-Modelle

V-Modell XT (Fortsetzg.): Vorteile

- Umfassender Standard
- Hochgradiges Tailoring
- Baukastenprinzip
- Weitgehend frei nutzbar
- Umfangreiche Arbeitshilfen verfügbar
- Frühzeitige Aufdeckung von Fehlern
- Klarer Prozessablauf

2.4) Vorgehensmodelle: V-Modelle

V-Modell XT (Fortsetzg.): Nachteile

- Großer Umfang auch noch *nach* Tailoring
- Signifikanter Verwaltungs-“Overhead“
- Teilw. ungewöhnliche Begrifflichkeiten

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzg.):
  - Einsatz-Empfehlung:
    - Bei hohen Qualitäts- bzw. Sicherheitsansprüchen an das Produkt
    - Für große Projekte
    - Ggf. Nutzung als Baukasten
    - Nach sorgfältig-kritischer Abwägung der Modell-Bestandteile
    - Standard im Öffentlichen Dienst
  - Grenzen: nicht enthalten sind insbes.
    - Vergabe von *Dienstleistungen* (◄► *Gewerke!*)
    - Betrieb und Aussonderung von IT-Systemen

2.4) Vorgehensmodelle: V-Modelle

- V-Modell XT (Fortsetzung):
  - Ausgewählte Änderungen im Rahmen der Release-Wechsel: u.A.:
    - Auslagerung des Projekttypen „Einführung und Pflege eines organisationsspezifischen Vorgehensmodells“ in ein eigenständiges V-Modell („V-Modell XT ORG“)
    - “Disziplinen” statt Produkt- bzw. Aktivitätsgruppen
    - Modifikationen bzgl. einiger Entscheidungspunkte
    - Modifikationen bzgl. Vorgehensbausteinen (insbes. Aufteilung der Vorgehensbausteine „Sicherheit …“)
    - Modifikationen bzgl. Produkten
    - Modifikationen bzgl. Rollen

2.4) Vorgehensmodelle: V-Modelle

- V-Modell 97, V-Modell XT: typ. Vertreter sog. plangetriebener / dokumentengetriebener / schwergewichtiger Modelle
- Weitere Vorgehensmodelle existieren, z.B.
  - Kommerzielle Modelle (z.B. RUP)
  - „Metamodelle“ (z.B. Spiralmodell)
  - Agile Modelle: im Folgenden / Kap. 2.5

2.5) Vorgehensmodelle: Agile Modelle

- Agile Modelle:
  - Entstanden als „Gegenbewegung“ zu bürokratisch-formalen Prozessen
  - „Manifesto for Agile Software Development“ (Agile Manifesto), 2001,

    „Principles behind the Agile Manifesto“

    (http://agilemanifesto.org)
  - Ausprägungen: XP, RAD, Prototyping, Scrum **u.A.**

2.5) Vorgehensmodelle: Agile Modelle

Agile Modelle (Fortsetzg.): Scrum

- Historie: Schwaber, Sutherland, seit ca. 1995
- Einsatzbereiche:
  - Produkt-Entwicklung
  - Management einer Aufgabenbearbeitung
- Basis:
  - Unmöglichkeit einer exakten frühen Planung (z.B. Zeit, Aufwand) bzw. Vorhersage der Ergebnisse
  - empirische Prozesssteuerung
  - → enges Controlling des Entwicklungsprozesses inkl. regelmäßige Rückkopplungen und Anpassungen
- Wichtigste Komponenten *gemäß K. Schwaber, J. Sutherland: „The Scrum Guide™“, Nov. 2020*:

2.5) Vorgehensmodelle: Agile Modelle

Agile Modelle (Fortsetzg.): Scrum

- *Scrum Theory:*
  - Empirie
  - Lean Thinking
  - Iterationen und Inkremente
  - *Transparency*
  - *Inspection*
  - *Adaptation*

- *Scrum Values:*
  - *Commitment*
  - *Focus*
  - *Openness*
  - *Respect*
  - *Courage*

2.5) Vorgehensmodelle: Agile Modelle

Agile Modelle (Fortsetzg.): Scrum

- *Scrum Events:*
  - *Sprint*
  - *Sprint Planning*
  - *Sprint Retrospective*
  - *Sprint Review*
  - *Daily Scrum*
- *Scrum Team:*
  - *Developers*
  - *Product Owner*
  - *Scrum Master*
- *Scrum Artifacts:*
  - *Product Backlog*
  - *Sprint Backlog*
  - *Increment**
- **Definition of Done*

2.5) Vorgehensmodelle: Agile Modelle

- Agile Modelle (Fortsetzg.): Scrum (Fortsetzg.)
- Übersicht:

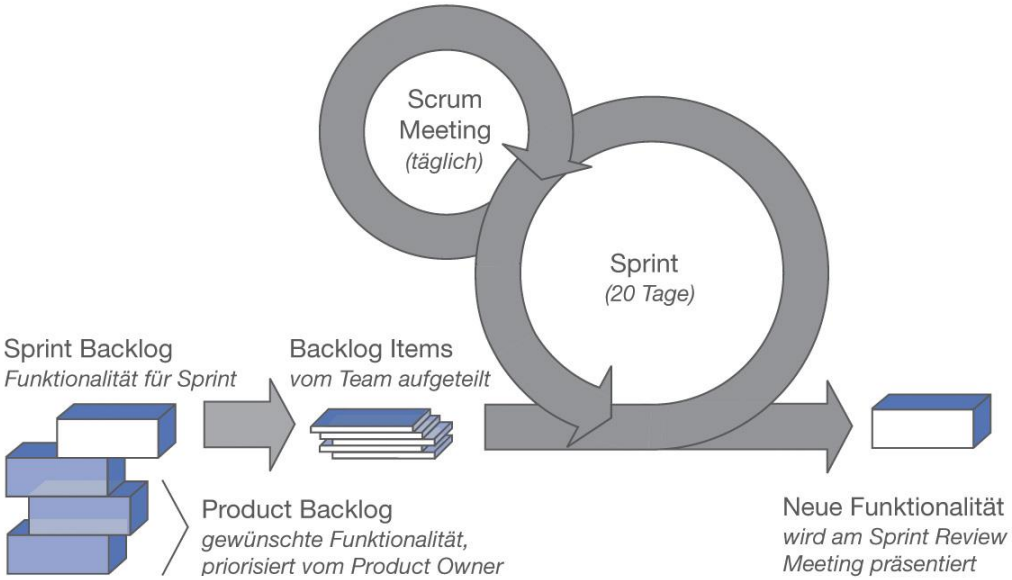


Abb. aus /Gre/

2.5) Vorgehensmodelle: Agile Modelle

Agile Modelle (Fortsetzg.): Scrum

- Vorteile: u.a.
  - Flexibilität bzgl. sich ändernder Anforderungen, Rahmenbedingungen etc.
  - Leichte Erlernbarkeit

- Nachteile bzw. Probleme: u.a.
  - Abhängigkeit von den Persönlichkeiten der Teammitglieder
  - Keine originäre Ausprägung bzgl. Software-Entwicklungen

- Änderungen gegenüber dem Scrum Guide Nov. 2017: u.a.
  - Wegfall des „Development Teams“
  - Einführung des „*Product Goal*“
  - Verschlankung und Kürzung des Dokumentes

2.5) Vorgehensmodelle: Agile Modelle

- Typ. Kontraindikationen für die Anwendung agiler Methoden: u.a.
  - Abweichende Vorgaben (z.B. Öffentl. Dienst)
  - Notwendigkeit für frühzeitige umfassende Planbarkeit
  - AG-seitige Schwierigkeiten
  - AN-seitige Schwierigkeiten
  - Größe bzw. Komplexität des Projektes bzw. der zu entwickelnden Lösung
  - Kritikalität der zu entwickelnden Lösung (z.B. bzgl. Security, Zertifizierungen)
  - Geringe Erfahrungen

2.5) Vorgehensmodelle: Agile Modelle

- Typ. Indikationen für die Anwendung agiler Methoden: u.a.
  - Notwendigkeit für Agilität
  - Bedarf für schnelle Auslieferung der (ersten) Inkremente
  - Entwicklung neuartiger Lösungen
  - Bewährte enge Zusammenarbeit AN-AG
  - AG-seitige Nutzenorientierung
  - AN-seitige Präferenz, „Neugier“ u.Ä. bei den Mitarbeitern

2.5) Vorgehensmodelle: Agile Modelle

- Abwägung agile vs. plangetriebene Modelle:

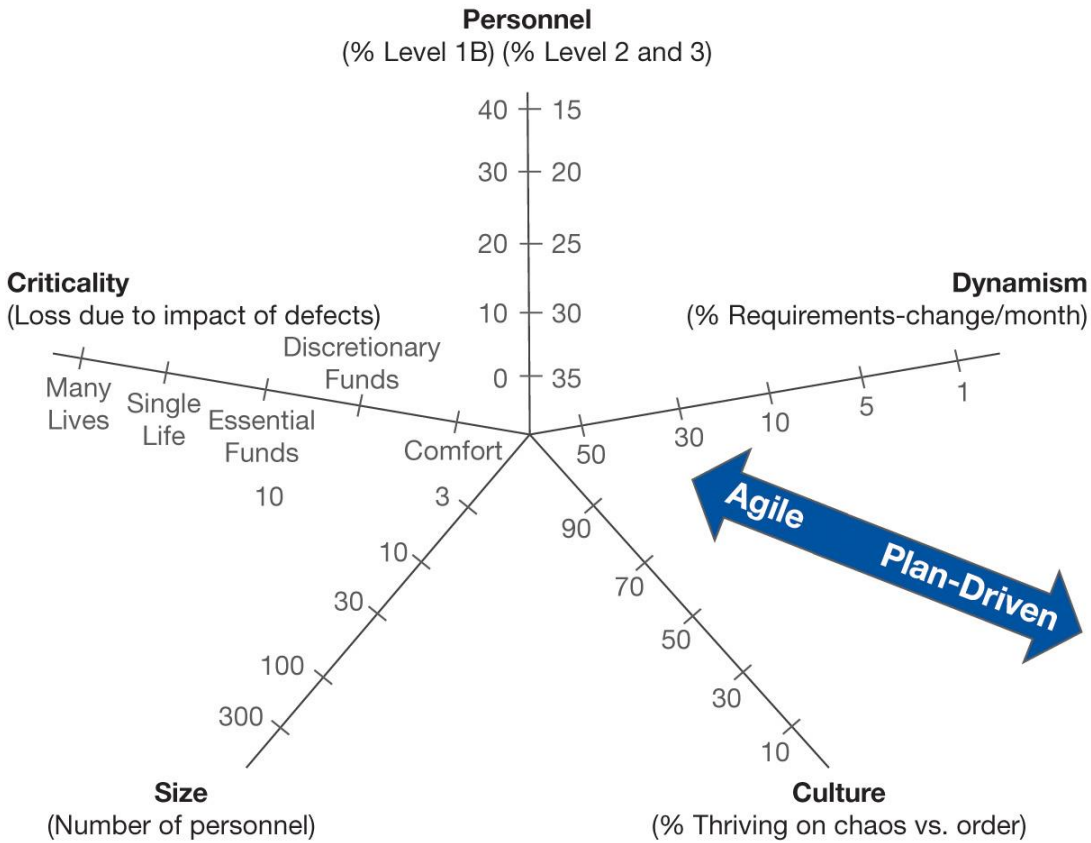


Abbildung 9.23: Entscheidungsmodell nach Boehm und Thurner (2003)

Abb. aus /Gre/

2.6) Vorgehensmodelle: Continuous Integration

- CI
- Ziele bzw. Vorteile:
  - Schnellere Generierung von Builds bzw. Inkrementen
  - Reduktion von Risiken insbes. durch
    - Vermeidung von “blind spots”
    - Geringere Integrations-Probleme
    - Geringere Rest-Fehlerraten
    - Höhere SW-Qualität
  - Basis für Continuous Delivery bzw. Continuous Deployment
- Achtung: Es gibt verschiedene Sprachgebräuche bzw. Abgrenzungen

2.6) Vorgehensmodelle: Continuous Integration

- Gemäß *M. Fowler „Continuous Integration“ (01-05-2006,* http://www.martinfowler.com/articles/continuousIntegration.html):

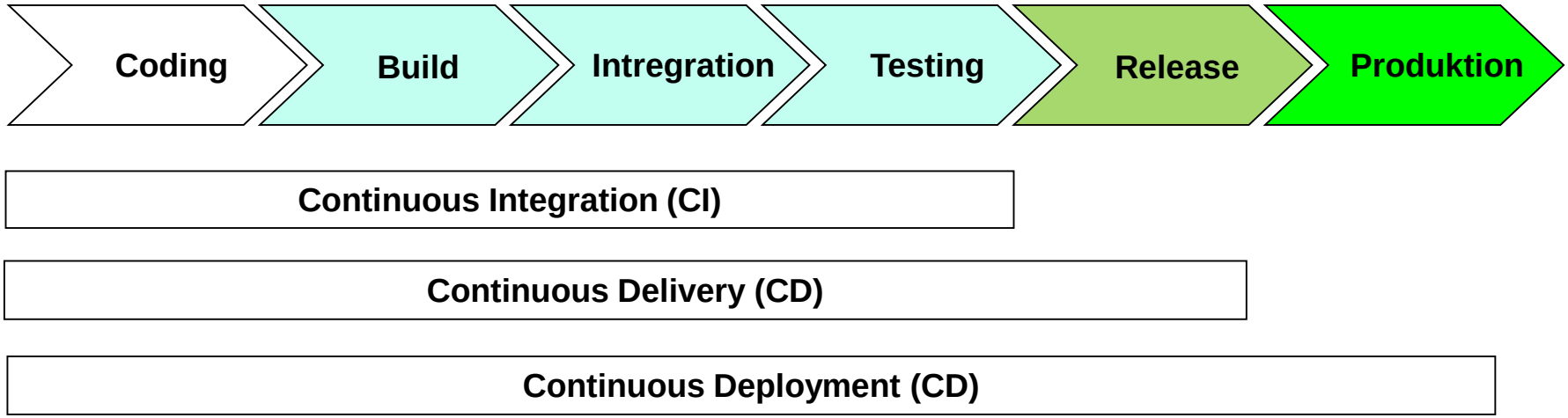
  „Continuous Integration is a software development practice where members of a team integrate their work frequently, usually each person integrates at least daily - leading to multiple integrations per day. Each integration is verified by an automated build (including test) to detect integration errors as quickly as possible. Many teams find that this approach leads to significantly reduced integration problems and allows a team to develop cohesive software more rapidly.”

2.6) Vorgehensmodelle: Continuous Integration

- *Key Practices* (gemäß o.g. Artikel von M. Fowler)
  - *Maintain a Single Source Repository*
  - *Automate the Build*
  - *Make Your Build Self-Testing*
  - *Everyone Commits To the Mainline Every Day*
  - *Every Commit Should Build the Mainline on an Integration Machine*
  - *Fix Broken Builds Immediately*
  - *Keep the Build Fast*
  - *Test in a Clone of the Production Environment*
  - *Make it Easy for Anyone to Get the Latest Executable*
  - *Everyone can see what's happening*
  - *Automate Deployment*

2.6) Vorgehensmodelle: Continuous Integration

- Pipeline:



3) Requirements Engineering / 3.1) Grundlagen

- Requirements Engineering: umfasst die 4 Haupttätigkeiten
  - Ermittlung
  - *Prüfung und Abstimmung/Festlegung*

    } Analyse
  - Dokumentation
  - Verwaltung

  der relevanten Anforderungen* an das Produkt und an das Projekt

  *: jeweils immer inkl. Rahmenbedingungen!

3) Requirements Engineering / 3.1) Grundlagen

- Relevanz:
  - „Maßstab“ für Qualität
  - Kunden-/Auftraggeber-Anforderungen sind die „Treiber“ der Software-Entwicklung; ihre Erfüllung ist die Basis für den Erfolg der Software
  - Anforderungen bilden Grundlage für Aufwandsschätzungen, Risikoanalysen, Auswahl des Vorgehensmodells etc., Codierung, Abnahmetests etc.
  - Viele Fehler (Boehm 1981: ca. 60%!) in Entwicklungsprozessen entstehen bereits im Requirements Engineering
  - Fehler im Requirements Engineering → Aufwand für Fehlerbehebung:
    - innerhalb des Requirements Engineering: sei Faktor 1
    - bei der Programmierung: ca. Faktor 20
    - in der Abnahmephase: ca. Faktor 100! (Boehm 1981)
- Fazit: Angemessenes Requirements Engineering ist ein MUST!

3) Requirements Engineering / 3.1) Grundlagen

Probleme:

- Umfang der Anforderungs-Bereiche ◄► Kunde formuliert primär seine *fachlich-funktionalen** Anforderungen
- Neben funktionalen Anforderungen: auch nicht-funktionale*!
- Neben expliziten Anforderungen: implizite Erwartungen und Rahmenbedingungen (z.B. Gesetze)
- Formulierungen i.d.R. umgangs- oder fachsprachlich
- Schwierigkeit der Konkretisierung oder Quantifizierung der Anforderungen
- Oftmals: genaue Anforderungen erst im Laufe des Projekts zu klären
- Oftmals: Änderung von Anforderungen im Laufe des Projektes

* (siehe z.B. ISO/IEC 9126-1, FURPS von HP)

3) Requirements Engineering / 3.1) Grundlagen

Analyse-Vorgehen: 1. Schritt: Ermittlung der Anforderungen

- Identifikation der „Quellen“
- Wichtig: Identifikation und Beteiligung der Stakeholder
- Hilfreich: Orientierung an Checklisten, Templates u.Ä.
- Unterscheidung „MUSTs“ ◄► „NICE-TO-HAVEs“ u.A.
- Ggf. zeitl. Priorisierung
- Gilt analog für ADDs, CHANGEs

3) Requirements Engineering / 3.1) Grundlagen

- Einschub: typ. Stakeholder = kompetente Vertreter der
  - Auftraggeber (i.w.S.)
  - Anwenderkreis(e)
  - IT-Abt.
  - QM / QS
  - Datenschutz-Einheit
  - Informationssicherheits-Einheit
  - GRC-Experten
  - Mitarbeitervertretungen
  - Ggf. Strategie-Einheit

  u.A.

3) Requirements Engineering / 3.1) Grundlagen

Einschub: Compliance-Anforderungen:

- Compliance = Erfüllung von gesetzl. Anforderungen, (Quasi-)Standards*, Richtlinien* u.Ä.
  * sowohl externe als auch unternehmens-interne
- Z.B. Gesetze:
  - Datenschutz: DSGVO, SGB, TKG
  - BWL: GoBS, GDPdU
  - Elektron. Signaturen: SigG, SigV

  u.v.a.m.

3) Requirements Engineering / 3.1) Grundlagen

Analyse-Vorgehen: 1. Schritt: Ermittlung der Anforderungen (Fortsetzg.)

- Wichtige sprachliche Aspekte bei der Aufnahme:
  - Klare Unterscheidung „muss“ (zwingend) ◄► „soll“ (empfohlen) ◄► „wird“ (zukünftig)
  - Vorsicht bei (Pseudo)Quantifizierungen wie „alle“, „ein“, „die“ (z.B. „die Daten“, „ein Anwender“)
  - Vorsicht vor Unvollständigkeiten bei Fallunterscheidungen
- Ggf. Einsatz von Prototypen

3) Requirements Engineering / 3.1) Grundlagen

Analyse-Vorgehen: 2. Schritt: Prüfung der Anforderungen

- Wichtig: Prüfung auf ggf.
  - unvollständige
  - unklare
  - widersprüchliche
  - widerstreitende
  - nicht angemessen erfüllbare

  Anforderungen

  → ggf. iterative Abstimmung mit Auftraggeber bzw. Stakeholdern!
- Prüf-Fragen: u.a.
  - Lassen sich aus den Anforderungen Test- bzw. Abnahmekriterien ableiten?
  - Tragen die Anforderungen zur Erfüllung der definierten Ziele bei?

3) Requirements Engineering / 3.1) Grundlagen

- Analyse-Vorgehen: 2. Schritt: Prüfung der Anforderungen (Fortsetzg.)
  - Prüfung der Konformität (z.B. Art der Dokumentation)
  - Insbes. auch bei Änderungen (ADDs, CHANGEs) von Anforderungen!

3) Requirements Engineering / 3.1) Grundlagen

- Analyse-Vorgehen: 3. Schritt: Abstimmung / Festlegung der Anforderungen
  - Ggf. Konsensfindung bzgl. der Stakeholder
  - Identifikation des / der Entscheidungsträger(s) seitens des Auftraggebers
  - Explizites Commitment / Abnahme durch den Auftraggeber
  - Dokumentation der Abnahme

3) Requirements Engineering / 3.1) Grundlagen

- Tätigkeit: simultane Dokumentation
  - Dokumentation der Anforderungen
  - Inkl. Anforderer
  - Inkl. Revisionsmanagement des Doks
  - Typ. im Lasten- bzw. Pflichtenheft
  - Strukturiert
  - Textuell und/oder mittels konzeptueller Modelle bzw. spez. Notationen: z.B. UML

3) Requirements Engineering / 3.1) Grundlagen

Zentrale Ergebnis-Dokumente (Fortsetzg.):

- Ergebnis-Dokument: wesentliche Inhalte:
  - Zielsetzung, möglichst auf Basis von IST und SOLL
  - Einsatzbedingungen (Anwendungsbereiche, Org.-Einheit, Zielgruppe, Betreiber des Produktes)
  - SOLL-Hauptfunktionen (insbes. auf der Basis von Use-Cases)
  - **Anforderungen an das Produkt**
  - Ggf. zusätzl. Anforderungen z.B. für die Entwicklungsumgebung, Tests u.Ä.
  - Meta-Anforderungen (z.B. an Projektmanagement, zusätzliche Dienstleistungen, Dokumentation)
  - Referenzen (insbes. auf mitgeltende Normen, Dokumente etc.)
  - Ggf.* Glossar (*sofern nicht separat)

3) Requirements Engineering / 3.1) Grundlagen

- Standards, Templates u.Ä.: u.a.
  - ANSI/IEEE-Std. 830-1998: IEEE Recommended Practice for Software Requirements Specifications: inkl. einiger Templates für SRS-Kapitel 3
    - SRS: Software Requirements Specifications
    - „Characteristics of a good SRS“:
      - Correct
      - Unambiguous
      - Complete
      - Consistent
      - Ranked for importance and/or stability
      - Verifiable
      - Modifiable
      - Traceable

3) Requirements Engineering / 3.1) Grundlagen

Standards, Templates u.Ä.: u.a.

- ANSI/IEEE-Std. 830-1998 (Fortsetzg.):
  - „The parts of an SRS“:
    1. Introduction
       - 1.1 Purpose
       - 1.2 Scope
       - 1.3 Definitions, acronyms, and abbreviations
       - 1.4 References
       - 1.5 Overview
    2. Overall description
       - 2.1 Product perspective
       - 2.2 Product functions
       - 2.3 User characteristics
       - 2.4 Constraints
       - 2.5 Assumptions and dependencies
    3. Specific requirements

    Appendixes

    Index

3) Requirements Engineering / 3.1) Grundlagen

Standards, Templates u.Ä.: u.a.

- ANSI/IEEE-Std. 830-1998 (Fortsetzg.):

  Zu 3. Specific requirements:

  3.1 External Interfaces

  3.2 Functions

  3.3 Performance Requirements

  3.4 Logical database requirements

  3.5 Design constraints

  3.6 Software system attributes (Reliability, Availability, Security, Maintainability, Portability)

3) Requirements Engineering / 3.1) Grundlagen

- Tätigkeit: Verwaltung der Änderungen: *u.a.*
  - Prüfung bzw. QS
  - Versionierung
  - „Verlinkung“ verschiedener, zugehöriger Informationen und Meta-Informationen
  - Änderungsmanagement!
  - Knowledge-Management

3) Requirements Engineering / 3.2) Agiles RE

- Typ. Aspekte agiler Vogehensmodelle: *u.a.*
  - Fokus auf den Kunden / AG
  - Durchgängige Zusammenarbeit von AG und AN
  - Wenig Dokumentation
  - Konkretisierung (Präzisierung, (zeitweilige) Finalisierung) der Anforderungen sukzessiv erst im Laufe des Projektes; i.d.R. auch dynamische Änderungen
    → RE ist entwicklungsbegleitend

3) Requirements Engineering / 3.2) Agiles RE

- Typ. Hilfsmittel: „User Stories“, Use Cases u.Ä.
- User Story:
  - “ … is a concise, written description of a piece of functionality that will be valuable to an user (or owner) of the software” *(K. Waters)*
  - U.a. Kern der Anforderung des Kunden in 1 - 2 Sätzen (umgangs-sprachlich, aktiv, in Geschäftsterminologie, aber typ. mittels Schablonen)

    → Konkretisierung erst im Laufe des Entwicklungsprozesses, iterativ in Zusammenarbeit zwischen AG und AN
  - Scrum: Autor ist Product Owner; User Story als Product Backlog Item
  - Fokus auf Funktionalität; non-funktionale Anforderungen i.d.R. separat als Constraints erfasst

3) Requirements Engineering / 3.2) Agiles RE

User Story (Fortsetzg.): Details:

- Zunächst primär als “Anforderungs-Themenspeicher” und als Zusage an den AG zur späteren Bearbeitung (“Versprechen für eine weitere Konversation über diesen Satz” *(Cockburn)*)
- Physische Form: Story Card, „Karteikarte“:

  Aufteilung z.B. („3 Cs“):

  - Card section (Vorderseite): *u.a.*
    - Id
    - User Story
    - Priorität
    - Geschätzter Aufwand
  - Conversation Section (Vorderseite):
    - Erläuterung der User Story
  - Confirmation section (Rückseite):
    - Akzeptanzkriterien (→ Akzeptanztests)

3) Requirements Engineering / 3.2) Agiles RE

User Story (Fortsetzg.): Details:

- Formulierung der Anforderung: typ. (Text-)Schablonen:
  - „As a [role] I want [something] so that [benefit]“ *(M. Cohn)*
  - „As a [user role], I want to [goal], so I can [reason]“ *(K. Waters)*
- Abgrenzung:
  - Ca. 1 - 3 Wochen Implementierungsdauer pro User Story
  - Übergeordnet: „Epics“
  - Abgeleitet: Tasks

3) Requirements Engineering / 3.2) Agiles RE

User Story (Fortsetzg.): Details:

- Qualitätskriterien / Eigenschaften:
  - Independent
  - Negotiable
  - Valuable
  - Estimable
  - Small
  - Testable

3) Requirements Engineering / 3.3) Ergänzung

- Zertifizerung:

  CPRE = „Certified Professional for Requirements Engineering“:

  - Gemäß IREB e.V. = International Requirements Engineering Board (seit 2007)
  - Schulungen durch anerkannte Ausbildungsunternehmen
  - Prüfungen und Zertifizierungen durch anerkannte Zertifizierungsstellen
  - Lit.: K. Pohl, C. Rupp: Basiswissen Requirements Engineering, dpunkt.verlag

4) Test von Software

Quellen-Hinweis:

Dieses Kapitel ist insbes. angelehnt an Teile der Vorlesung „Software Testmethoden I“ von Herrn H. Blanke-Bohne,

jene wiederum orientiert sich u.a. am Buch „Basiswissen Softwaretest“ von A. Spillner und T. Linz, dpunkt.verlag = /Spi/

jenes wiederum am Lehrplan zum „ISTQB Certified Tester – Foundation Level“ des ISTQB / International Software Testing Qualification Board (Syllabus Version 2011)

4.1) Einführung

- Berufsbild, Fortbildungen
- ISTQB = International Software Testing Qualification Board:
  - mehrere Level
  - Zertifizierung durch iSQI (International Software Quality Institute)
  - > 100.000 Testing Professionals

4.1) Einführung

- Grundbegriffe:
  - Testen:
    - „Unter dem Testen von Software wird jede (im Allgemeinen stichprobenartige) Ausführung eines […] Testobjekts verstanden, die der Überprüfung des Testobjektes gilt.“ (aus /Spi/)
    - „Der Prozess beinhaltet alle statischen und dynamischen Aktivitäten des Lebenszyklus, die sich mit der Planung, Vorbereitung und Bewertung eines Softwareproduktes (und der zugehörigen Arbeits-ergebnisse) befassen. ….“ (aus /Spi/)

4.1) Einführung

Relevanz:

- Grundlage für Abnahmen, vertrauensvollen Einsatz, Zertifizierungen u.Ä. der Software
- Praxis:
  - Software-Erstellung ist fehlerträchtig ⇒ in Entwicklungsphase: Testaufwand ≈ typ. 25 – 50%* des Gesamt-Entwicklungsaufwandes

    *: und ggf. mehr!
  - Software ist i.d.R. auch nach der Freigabe fehlerbehaftet ⇒ in Wartungsphase: Aufwand zur Fehlerbehebung ≈ typ. 20% des Gesamt-Wartungsaufwandes
  - *Faustformel*: Entdeckungsrate der Fehler durch Tests ≈ typ. etwa 50% ⇒ Wichtigkeit der Fehler-Vermeidung!

4.1) Einführung

Grundbegriffe (Fortsetzg.):

- Debugging: Ziel: Lokalisation + Analyse + Behebung des Fehlerzustandes (◀▶ Testen insbes. zur Feststellung von Fehlerwirkungen)
- Testmanagement: Planung, Durchführung und Auswertung der Tests
- Testobjekt: zu testendes „System“ (z.B. Programm, Modul)
- Testfall: u.a.
  - festgelegte Randbedingungen:
    - Voraussetzungen zur Ausführung
    - Eingabe-/“Test“werte
    - erwartete Ausgabe / erwartetes Verhalten (= Sollvorgabe)
  - Ziel:

    Aufdeckung bisher unbekannter Fehlerwirkungen
- Testlauf: Ausführen von einem oder mehreren Testfällen

4.1) Einführung

Grundbegriffe (Fortsetzg.):

- Testbasis: alle relevanten Infos bzgl. des Testobjektes, z.B.
  - Anforderungen
  - Spezifikationen und Design
  - Architektur und Schnittstellen
  - Struktur
  - Sourcecode

4.1) Einführung

Testaufwand:

- Vollständiges Testen ist i.d.R. nicht möglich
- Notwendiger Testaufwand:
  - ist immer Projekt-abhängig, u.a.: von
  - Risiken bei Fehlern: z.B.
    - Absatzverlust
    - Produktionsausfall
    - Strafen
    - Gefährdung von Personen oder Umwelt
- Notwendigkeit zur Beschränkung des Aufwandes ⇒ systematisches Testen, insbes. Auswahl geeigneter Testverfahren und Testfälle

4.1) Einführung

Typ. Problembereiche:

- Testobjekt-inhärente Aspekte: u.a.
  - Komplexität
  - Mangelnde Informationen
- Umgebungsbezogen-technische Aspekte: u.a.
  - Nicht-Verfügbarkeit einer angemessenen Testumgebung
  - Nicht-Verfügbarkeit von adäquaten Testdaten
- Projektbezogene bzw. betriebswirtschaftl. Aspekte: u.a.
  - Fehlende Awareness bzgl. der Relevanz
  - Unangemessene Klärung der Anforderungen
  - Unangemessene Ressourcen (oftmals Fokus auf Codierung)
  - Unangemessenes Test-Management

4.1) Einführung

Typ. Problembereiche (Fortsetzg.):

- Psychologische Aspekte: u.a.
  - „Errare humanum est“
  - Nicht-Erkennen eigener Fehler
  - Fehlerkultur

4.2) Fundamentaler Testprozess

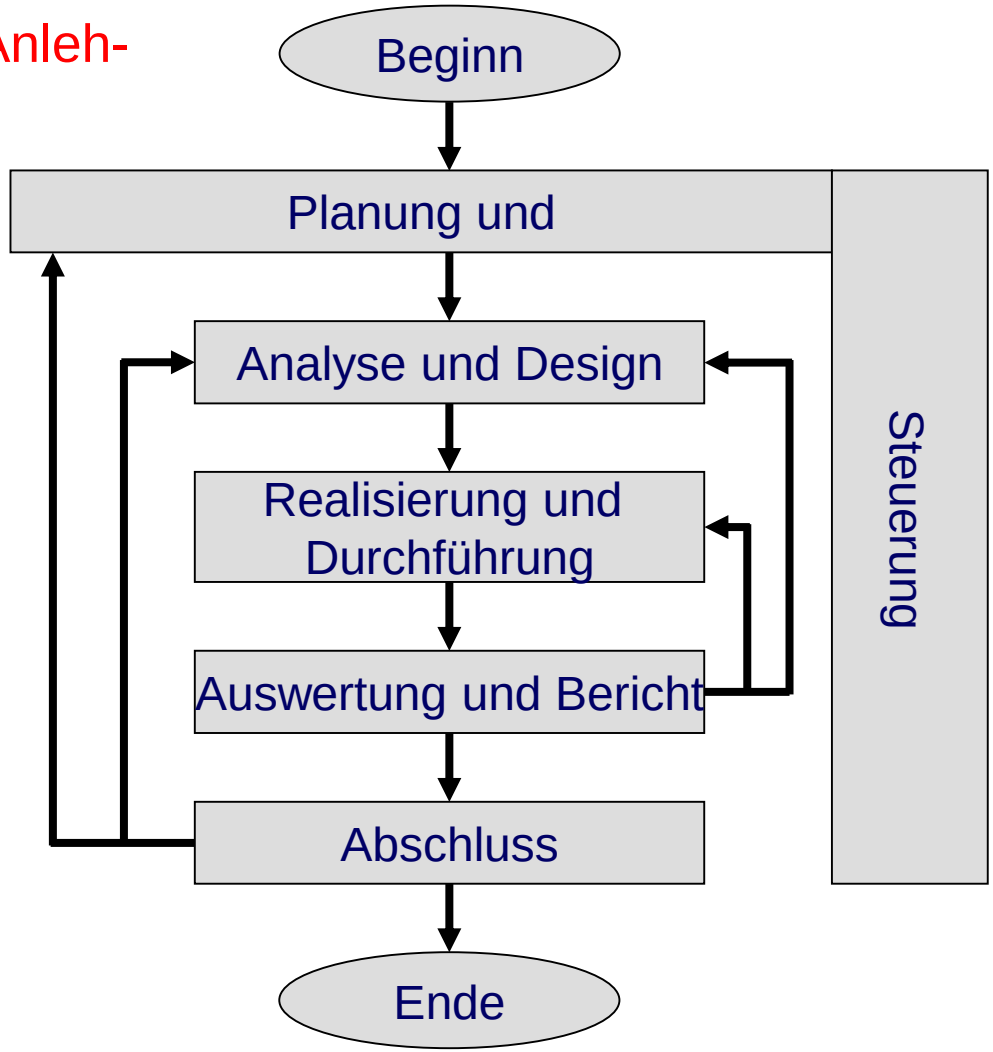
- Testen: bestehend aus div., geordneten Phasen, Aktivitäten u.ä. Komponenten

  → allg. Ablaufplan für die Testarbeiten:

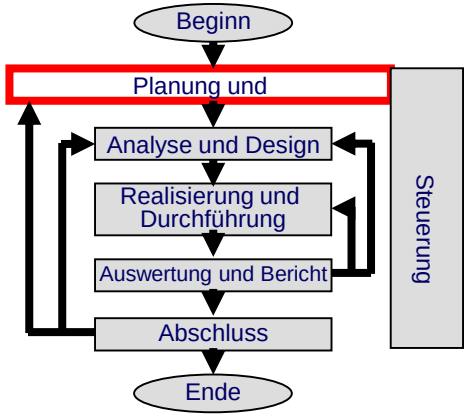
- Bsp.: ISTQB Fundamentaler Testprozess (Version 2011) ← Fokus im Folgenden

4.2) Fundamentaler Testprozess

- Testprozess in Anlehnung an ISTQB (Version 2011):



4.2) Fundamentaler Testprozess



- Teil-Phase Testplanung: insbes.
  - Festlegung der Zielsetzung und Aufgaben
  - Planung der Ressourcen: u.a.
    - Team / Mitarbeiter mit Rollen, Verantwortlichkeiten, Qualifikationen und Skills
    - Zeit etc.
  - Festlegung der Teststrategie: u.a.
    - Festlegung der Testmethoden, Testintensitäten, Überdeckungsgrade
    - Vorbeugend ….. reaktiv
    - Analytisch ….. heuristisch
    - Risikoanalyse
    - Verteilung der Tests auf Teilsysteme

4.2) Fundamentaler Testprozess

- Teil-Phase Testplanung (Fortsetzg.): insbes.
  - Priorisierung der Tests
  - Festlegung des Testende-Kriteriums

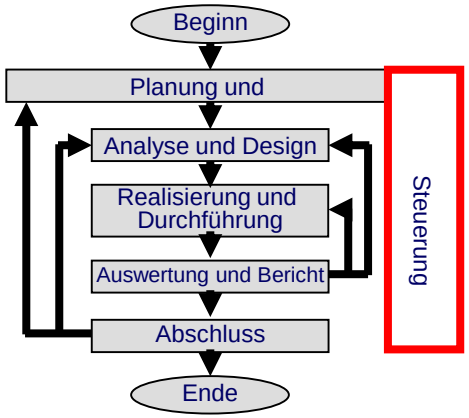
  → Testkonzept / -plan
  - Auswahl, Beschaffung bzw. Entwicklung von Werkzeugen u.Ä. (Testware, Testrahmen (Test Bed, Test Harness))

4.2) Fundamentaler Testprozess

Exkurs: Testrahmen

- Testtreiber: spezif. Programm zur automat. Aufruf des Testobjektes inkl. I/O
- Stub (Mock, Dummy): Platzhalter für noch nicht verfügbare, benötigte Komponenten
- Laufzeitumgebung, Monitore, Debugger:
  z.B. zur Beobachtung von Programmvariablen

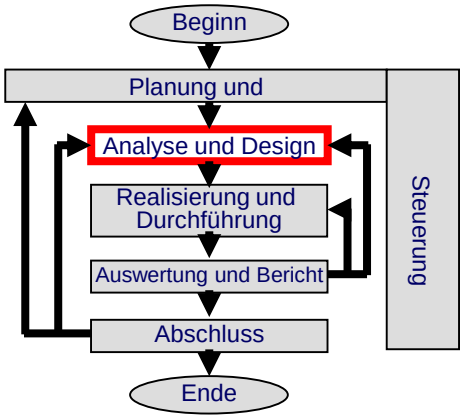
4.2) Fundamentaler Testprozess



- Teil-Phase Teststeuerung: insbes.
  - Fortlaufende Überwachung der Testaktivitäten und des Testfortschrittes
  - Fortlaufende SOLL-IST-Vergleiche

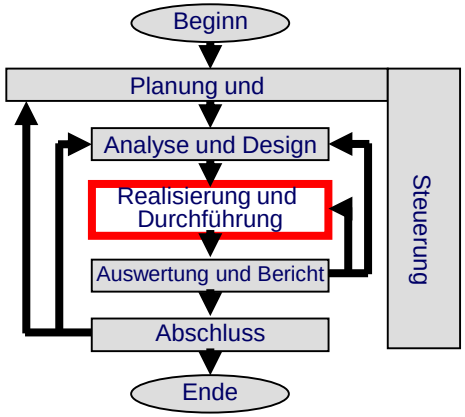
    → bei Abweichungen:
    - Kommunikation
    - Einleitung von Korrekturmaßnahmen
    - Ggf. Aktualisierung der Planung
  - Fortlaufendes Konfigurations- und Change-Management
  - Verwaltung der Testinfrastruktur und Testware

4.2) Fundamentaler Testprozess



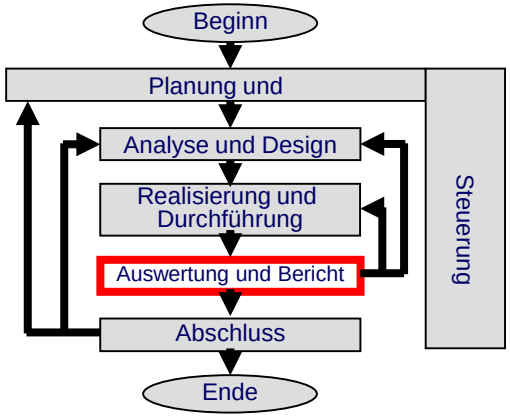
- Phase Testanalyse und -design (Fortsetzg.): insbes.
  - Ableitung der Testfälle aus der Testbasis
  - Entwurf der Testfälle
  - Pro Testfall:
    - Traceability des Testfalls zur Anforderung (und umgekehrt)
    - Vorbedingungen / Voraussetzungen und Randbedingungen
    - Festlegung der Eingaben
    - Soll-Ergebnis*:
      - Erwartete Ausgabe
      - Erwartetes Verhalten
      - Erwartete Änderungen
    - Nachbedingungen
  - * Ableitung insbes. aus:
    - Spezifikation
    - Inversion

4.2) Fundamentaler Testprozess



- Phase Testrealisierung und -durchführung: insbes.
  - Endgültige Realisierung *und Test* (!!!) der Testware und Testinfrastruktur (Achtung: Fehlerwirkungen sind möglich durch Fehlerzustände in der Testware!)
  - Einbindung des Testobjektes in die Testware
  - Festlegung der *konkreten* Testfälle und -sequenzen
  - Durchführung von Probe-Testläufen
  - Durchführung der exakten Testfälle* *inkl. Protokollierung bzw. Fehlermanagement*
  - * Empfehlung: Hauptfunktionalitäten *zuerst* testen

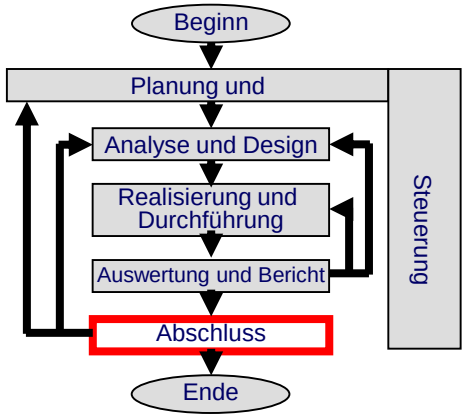
4.2) Fundamentaler Testprozess



➢ Phase Testende-Auswertung und Bericht:

- Bewertung, ob die geplanten Testendekriterien erfüllt sind ⇒
  - Falls ja: Beendigung der Testaktivitäten
  - Falls nein: Fortsetzg. der Aktivitäten, ggf. Entscheidung, dass zusätzliche Testfälle erforderlich sind oder Modifikation der Kriterien, oder aber ggf. Beendigung der Testaktivitäten
- Detail Testendekriterien: typ.
  - Erreichte Testabdeckung
  - Erreichte Produktqualität (z.B. Fehlerschwere, Fehlerfindungsrate)
  - Restrisiken
  - Rahmenbedingungen (z.B. Kosten)
- Bericht: Zusammenfassender Bericht für Stakeholder

4.2) Fundamentaler Testprozess



Phase Testabschluss

- Konservierung der Testware, denn:
  - alle Tests, Testrahmen und Dokumente aus dem Testprozess können ggf. während der Wartung und Pflege wieder benutzbar sein, z.B. für Regressionstests)
  - evtl. auch für ähnliche Projekte verwendbar
- Evaluation des Testprozesses:
  - Kritischer Rückblick auf den Testprozess, z.B. Abweichung zwischen Planung und Umsetzung?
  - Fragen: z.B.
    - Welche geplanten Ergebnisse wurden wann erreicht?
    - Welche unvorhergesehenen Ereignisse gab es?
    - Wie war die Akzeptanz beim Einsatz des Systems?

→ Ableitung von Optimierungspotentialen für weitere Projekte

4.3) Grundlegende Testarten

- Tests werden unterschieden / kategorisiert z.B.:
  - Je nach Zweck: funktionale Tests ◄► nicht-funktionale Tests (z.B. Last- bzw. Stresstest)
  - Je nach Durchführenden (z.B. Entwickler- oder Anwendertests)
  - Je nach Stufe im Entwicklungsprozess (z.B. Modul- oder Abnahmetest)
  - Je nach Umfang (z.B. GUI-Tests)
  - Blackbox- ◄► White- / Glasbox- ◄►Greybox-Tests
- Ausgewählte spezielle Tests:
  - Änderungsbezogene Tests
  - Statische ◄► dynamische Tests

4.3) Grundlegende Testarten

Änderungsbezogener Test: Regressionstest

- Test nach *Änderung** der (bereits getesteten bzw. eingesetzten) Software
- *Änderung: z.B. in Form von Debugging, Erweiterung, Migration
- Ziel: u.a. Aufspüren von Folgefehlern durch die Modifikation
- Durchführung:
  - Testen der geplant *unveränderten* Eigenschaften (z.B. Funktionalitäten):
    - mit alten Echtdaten → Vergleich mit Echtbetrieb-Ergebnissen
    - oder Wiederholung früherer Tests mit erfolgreichem Ausgang → Vergleich mit früheren Testergebnissen
  - Testen der geplant *geänderten* Eigenschaften
  - Wiederholung der Tests, die Fehlerwirkungen aufgezeigt haben
  - Testen des kompletten Systems
- Praxis:
  - i.d.R. sehr häufiger Bedarf (Entwicklung, Wartung!) → Einplanung + proaktive Maßnahmen (z.B. Archivierung früherer Tests)
  - Test des kompletten Systems wird oftmals unterlassen wg. Aufwand → Auswahl der Testfälle

4.3) Grundlegende Testarten

- Änderungsbezogener Test: Regressionstest (Fortsetzg.)

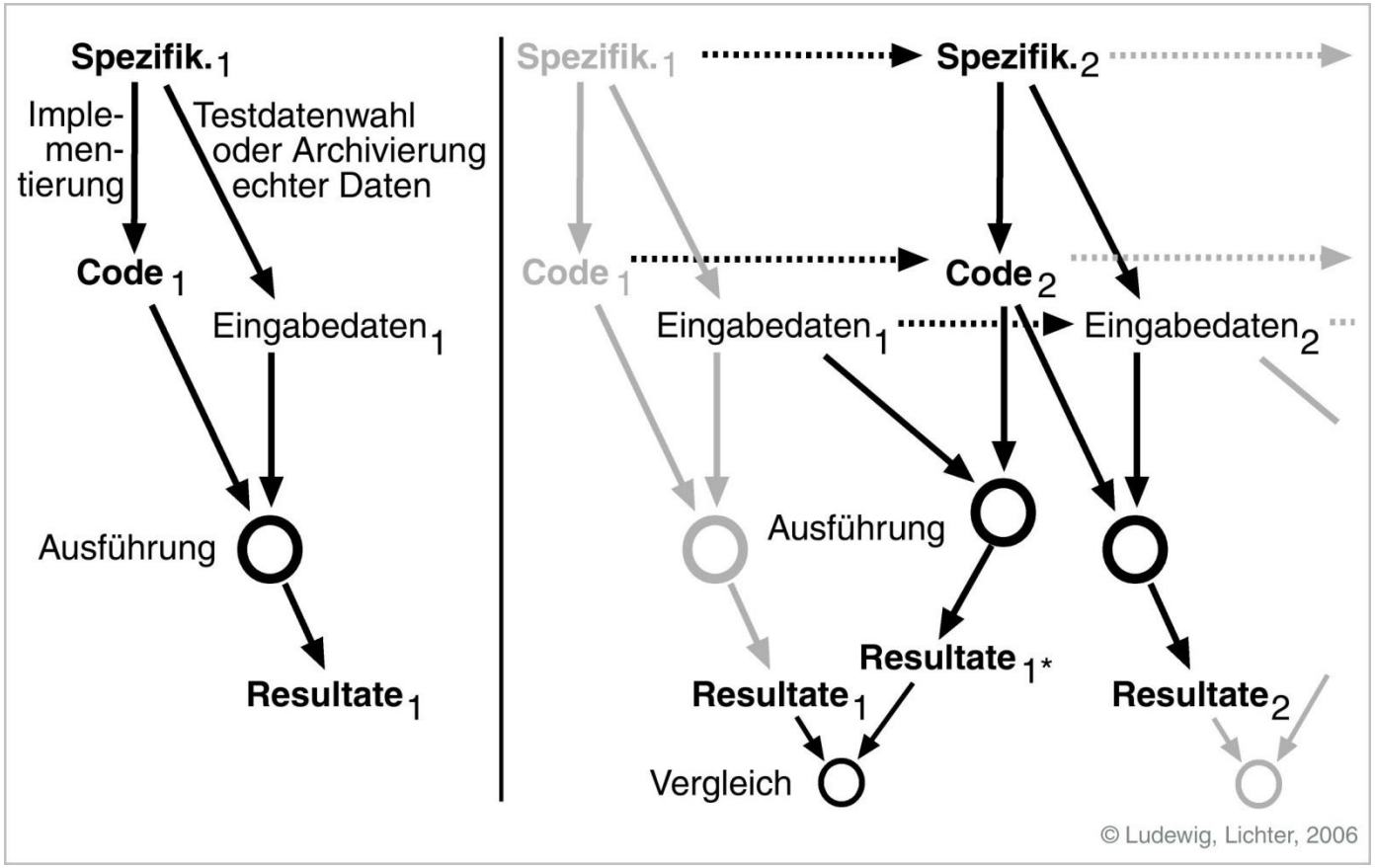


Abb. aus /Lud/

4.3) Grundlegende Testarten

- Statischer Test :
  - Analysieren des Prüfobjekts (Dokument oder Sourcecode o.Ä.)
  - Ohne *autom. Ausführung* des Testobjekts
  - Mittels Reviews, statische Analyse
- Dynamischer Test:
  - *Autom. Ausführung* des Testobjekts auf einem Testsystem
  - Testobjekt braucht i.d.R. Eingabedaten, läuft im Testrahmen

4.3) Grundlegende Testarten

- Blackbox-Tests:
  - keine Infos über Sourcecode und inneren Aufbau
  - Steuerung des Ablauf nur durch Wahl der Eingabedaten
  - Testfälle beruhen auf Spezifikation und Anforderungen an das Testobjekt
- Whitebox- / Glasbox-Tests: (strukturelle Verfahren)
  - Zugriff auf den Sourcecode, Analyse des inneren Ablaufs
  - in Ausnahmefällen: Eingriff in das Testobjekt
    (z.B. zwecks Detail-Analysen des Ablaufs im Testobjekt)
  - Testfälle können aufgrund der Programmstruktur gewonnen werden

5) Spezifikationstechniken

5.1) Objektorientierung - Einführung

- Historie:
  - Seit 1980: OOP ← Smalltalk-80 (← Simula-67)
  - → seit Anfang der 1990er Jahre: C++, Java (1996), C# (2001)
  - Seit Ende der 1980er Jahre: OOA, OOD (Booch, Rumbaugh, Jacobson; Coad u. Youdan; Shlaer u. Mellor u.A.)
  - Seit ca. Mitte der 1990er Jahre: UML

5) Spezifikationstechniken

5.1) Objektorientierung - Einführung

➢ Grundkonzept (vereinfacht) des OOx:

- Orientierung an
  - abgrenzbaren, realen Objekten („Objekt“) mit ihren Eigenschaften (→ „Attribute“)
  - deren verallgemeinernde / abstrahierende Arten / Schablonen (→ „Klassen“)
  - deren Taxonomie (→ „Oberklasse“ bzw. „Unterklasse“) und Zusammenhang (→ „Vererbung“ u.ä.)
  - den möglichen Operationen (→ „Methoden“)
  - der Wechselwirkung zwischen Objekten

  etc.

5) Spezifikationstechniken

5.1) Objektorientierung - Einführung

- OOx:

OOSE

OOA (OOAnalyse):
- OO im Problembereich:
- Identifizierung der Objekte, Zusammenhänge etc.

OOD (OODesign):
- OO im Lösungsbereich:
- Entwurf OOD-Modell / Fachkonzept (ggf. separat: GUI und Datenhaltung)
- Ggf. Überarbeitung der Objekte

OOP (OOProgrammierung):
- OO im Codebereich:
- Codierung mittels OO-Programmiersprache

- Lifecycle: OOA → OOD → OOP
- Besonderer Vorteil: einfache Ableitbarkeit der Phasen
- Im Folgenden: Beschränkung insbes. auf OOA und UML

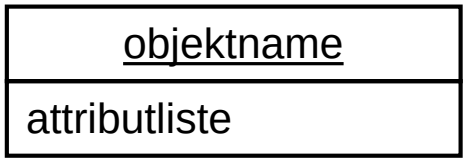
5) Spezifikationstechniken / 5.1) OO-Einführung

- Groß- bzw. Kleinschreibung (Capitalization):

Groß/Kleinschreibung	Beispiel	Beschreibung
Standard C / Gnu Notation	mouse_weight	Alles kleingeschrieben, Trennung von zusammengesetzten Worten durch Unterstrich.
Ungarische Notation	iMouseWeight	Kombination aus Typ + Bezeichner.
Camel	mouseWeight	Erstes Wort kleingeschrieben, alle weiteren Teilbezeichner werden mit einem Großbuchstaben gekennzeichnet.
Pascal	MouseWeight	Der jeweils erste Buchstabe eines Bezeichners und aller folgenden Teilbezeichner wird großgeschrieben.
„All capitals“	MOUSE_WEIGHT	Alle Elemente werden großgeschrieben. Teilbezeichner werden über Unterstriche miteinander verbunden.

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

- Objekt:
  - = konkreter, abgrenzbarer, „realer“ Gegenstand des Interesses
  - Charakterisiert durch:
    - Bezeichnung
    - Zustand (state) mit Attributen und aktuellen Werten sowie Beziehungen (links) zu anderen Objekten
    - Verhalten (behavior)
  - 

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Objekt (Fortsetzg.):

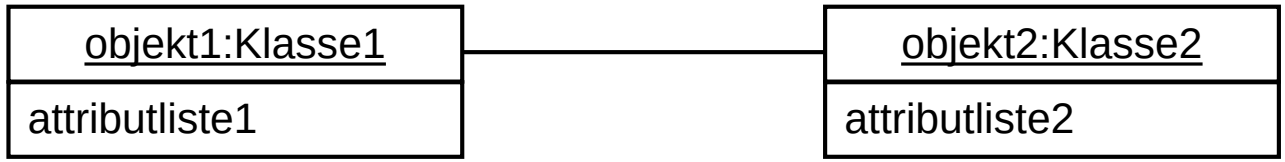
- Objektname:
  - objekt:Klasse ◄► objekt ◄► :Klasse
  - Relevanz nur innerhalb 1 Diagramms
- Attributliste: attribut, Typ, Wert o.a.
- Daten können nur mittels (zugelassener) Operationen geändert bzw. gelesen werden
- Geheimnisprinzip: Objekte können ihre Daten und Operationen kapseln / verbergen

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

➢ Objekt (Fortsetzg.):

- Objektdiagramm:



- Objektidentität (keine 2 Objekte besitzen dieselbe Identität!)

  ◄► Objektgleichheit (n Objekte mit gleichen Attributwerten)

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Klasse:

- Objekte mit den *gleichen* Attributen und Operationen bilden eine Klasse
- Zugehörige Objekte sind „Exemplare“ bzw. „Instanzen“ der Klasse; jedes Objekt „kennt“ seine Klasse (mit Operationen)
- Klasse definiert für eine Gruppe an Objekten:
  - Attribute
  - Operationen
  - Beziehungen
-

Klassenname
attribut: Typ
operation()

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Klasse (Fortsetzg.):

- Klassendiagramm
- *Abstrakte* Klasse
  - = Klasse, die nicht instanziierbar ist
  - Notation:
    - *Klassenname kursivgeschrieben* oder
    - {abstract} Klassenname

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

- Attribute:
  - attribut
    - ◄► attribut: Typ
    - ◄► attribut: Typ {Eigenschaftswert}
    - ◄► attribut: Typ = Wert
    - ◄► attribut: Typ [Multiplizität]
  - <u>klassenattribut</u> (class scope attribute)
  - Einschränkungen (constraints):
    - Sind zulässig
    - Notation: {Einschränkung}
  - /abgeleitetes Attribut
  - Attributname nur innerhalb der Klasse eindeutig: außerhalb: Klasse.attributname

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Operationen:

- Auf alle Objekte einer Klasse sind die Operationen anwendbar
- Jede Operation kann auf jedes Attribut zugreifen
- „Verhalten“ der Klasse = Menge aller Operationen einer Klasse
- Arten: insbes.
  - Objektoperationen / Operationen: wirken auf einzelnes Objekt
  - Konstruktoroperationen: erzeugen und initialisieren ein neues Objekt
  - <u>klassenoperationen</u>:
    - Operation ohne Zuordnung zu Objekten
    - Operation mit (gleichzeitiger) Wirkung auf mehrere oder alle Objekte der Klasse

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Operationen (Fortsetzg.):

- Notation:
  - Verb (Kleinschreibung) und ggf. Substantiv
  - Als Ende: ()
- Nur Operationen ändern die internen Daten eines Objektes
- Objekt führt Operation nur dann aus, wenn es eine Nachricht / Anfrage / Meldung (message) von einem Klienten erhält

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

- Beziehungen zwischen Objekten:
  - Synonym: Link, Objektbeziehungen
- Assoziation
  - = Menge aller Beziehungen zwischen *Objekten* gleichrangiger Klassen bzw. zwischen Objekten der selben Klasse (reflexive Assoziation)
  - Assoziation: im Klassendiagramm eingetragen
  - Notation: siehe folgende Seite; ggf. mit Angabe der Multiplizität am zugehörigen Linien- bzw. Pfeilende
  - Multiplizität: konkrete Zahl ◄► Wertebereich .. ◄► unbestimmt viele: *
  - Kann-Assoziation: Multiplizitäts-Untergrenze = 0
  - Muss-Assoziation: Multiplizitäts-Untergrenze ≥ 1
  - Assoziationen können benannt werden; dabei „Leserichtung“ mittels ► kennzeichnen

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

➢ Assoziation (Fortsetzg.):

- Navigierbarkeit und Notation:
  - Standard-UML:
    - Unidirektional: ⟶
    - Bidirektional: ⟷
    - Nicht-Navigierbarkeit: ✕⟶
      (hier: am linken Ende)
    - Unspezifiziert / undefined: ——

◀ ▶

  - Praxis / oft:
    - Unidirektional bzw. Nicht-Navigierbarkeit: ⟶
    - Bidirektional: ——

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Assoziation (Fortsetzg.):

- Rollen- bzw. Assoziationsnamen:
  - Rollenname: Kennzeichnung der „Bedeutung“ der Objekte für die Klassen
  - Rollen- bzw. Assoziationsnamen: müssen angegeben werden, falls ≥ 1 Assoziation zwischen den bzw. der selben Klassen besteht
- Ggf. {Eigenschaftswert} oder {Einschränkungen} anhängbar an Assoziationsnamen
- Assoziation kann Attribute, Operationen und Assoziationen zu anderen Assoziationen besitzen → Assoziationsklasse: besitzt Eigenschaften einer Klasse und einer Assoziation

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Assoziation (Fortsetzg.):

- Arten:
  - Einfache Assoziation (ordinary association)
  - n-äre Assoziation (z.B. binäre Assoziation = Assoziation zwischen 2 Objekten)
  - Aggregation
  - Komposition
- n-äre Assoziation:
  - Assoziation zwischen mehr als 2 Klassen
  - Notation: u.a. Raute zwischen den Klassen, von dem aus die Linien zu den Klassen gehen
  - Abgeleitete Assoziation = Assoziation, die durch (die Zusammenfassung mehrerer) andere Assoziationen abgeleitet werden kann

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Assoziation (Fortsetzg.):

- Aggregation:
  - falls zwischen den Objekten der Klassen Rangordnung („ist Teil von“, „besteht aus“ o.ä.***) besteht
  - Gerichteter Graph
  - Notation: unausgefüllte Raute am Linienende beim Ranghöheren (Aggregat-Klasse, „Ganzes“)
  - Achtung: „Teil“ kann auch alleine existieren!
  - *** Achtung: Sprachgebrauch oftmals nicht eindeutig! Möglichkeit der Komposition prüfen!
- Komposition (composition, composite aggregation):
  - Spezialfall der Aggregation: Multiplizität an der Aggregat-Klasse = 1
  - Notation: *gefüllte* Raute (mit „1“) am Linienende beim Ranghöheren
  - Achtung: Beim Erzeugen bzw. Löschen des Aggregat-Objektes werden auch die „Teile“ erzeugt bzw. gelöscht! „Teil“ kann nicht alleine existieren!

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Generalisierung und Spezialisierung:

- Generalisierungsstruktur bzw. Klassenhierarchie
- Hierarchische Beziehung zwischen einer Basisklasse / Oberklasse / Superklasse (super class) und einer spezialisierten Klasse / Subklasse / abgeleiteten Klasse (sub class)
- Generalisierung: Ableitung einer Oberklasse aus Klassen
- Spezialisierung: Ableitung einer Subklasse aus einer Oberklasse
- Ggf. „Hierarchieketten“

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Generalisierung und Spezialisierung (Fortsetzg.):

- Vererbung (inheritance): Weitergabe aller Eigenschaften, hier von der bzw. den Oberklasse(n) auf die Subklassen
- Subklasse hat alle Eigenschaften der Oberklasse(n), aber zusätzlich weitere Attribute, Operationen, Assoziationen!
- Operation der Oberklasse kann in Subklasse modifizert (redefined) werden
- Finale Klasse = Klasse, die nicht weiter spezialisierbar ist
- Notation: unausgefülltes Dreieck am Übergang von der Klassenassoziations-Linie zur Oberklasse

5) Spezifikationstechniken

5.2) OO: Objekte und Klassen

Generalisierung und Spezialisierung (Fortsetzg.):

- Vererbung:
  - Ja: Attribute, Operationen, Klassenoperationen, Klassenattributwert, Assoziation
  - Nein: Attributwerte, Konstruktoren
- Vorteile:
  - Einfache Veränderbarkeit durch direkte Änderung nur der Oberklasse
- Nachteile bzw. Probleme:
  - Ggf. umfassende Auswirkungen von Änderungen in Oberklassen
  - Verständlichkeit von Subklassen „tief“ in der Generalisierungsstruktur

5) Spezifikationstechniken

5.3) OO: UML-Ergänzungen

- UML = Unified Modelling Language
- Einsatzzweck: standardisierte Notation bzgl. OOA, OOD
- Historie:
  - Booch, Rumbaugh, Jacobson: UML 0.9, 1996
  - UML 1.0: 1997
  - → OMG (Object Management Group, www.omg.org)
  - UML 2.0: 2005
  - Aktuell: UML 2.5
- Normen:
  - ISO/IEC 19501:2005: UML 1.4.2
  - ISO/IEC 19505-1, -2:2012 UML 2.4.1

5) Spezifikationstechniken

5.3) OO: UML-Ergänzungen

- UML definiert:
  - Bezeichner für Begriffe
  - die Beziehungen zwischen den Begriffen
  - Grafische Notationen für die Begriffe und Modelle
- → Diagramme
- (OCL / Object Constraint Language, zur textuellen Darstellung von versch. Zusicherungen bzw. Einschränkungen)

5) Spezifikationstechniken

5.3) OO: UML-Ergänzungen

- Diagramm-Typen:
  - 7 Strukturdiagramme (structure diagrams):
    - Klassen-Diagramm
    - Objekt-Diagramm
    - Paket-Diagramm
    - Komponenten-Diagramm
    - Kompositionsstruktur-Diagramm
    - Einsatz- und Verteilungs-Diagramm
    - Profil-Diagramm
  - 7 Verhaltensdiagramme (behaviour diagrams):
    - Anwendungsfall-/Use-Case-Diagramm
    - Aktivitäts-Diagramm
    - Zustands-Diagramm
    - Sequenz-Diagramm
    - Kommunikations-Diagramm
    - Interaktionsübersichts-Diagramm
    - Zeitverlaufs-Diagramm

    } Interaktions-Diagramme

Schlusswort

- Schwerpunkte / Themenbereiche:
  - Einleitung: Begriffe, Motivation, Software-Lebenszyklus
  - Vorgehensmodelle
  - Requirements Engineering
  - Testen von Software
  - Spezifikationstechniken (OO, UML)

- → Software-Engineering: ein vielfältiges, aber existenziell wichtiges Gebiet zur Entwicklung und Betrieb erfolgreicher Software!

Schlusswort

Vielen Dank

für Ihre

Mitarbeit und

Aufmerksamkeit!